

An Exponential Succinctness Gap between Three-Variable Logic and the Calculus of Relations

Yuya Uezato

CyberAgent, Inc., uezato_yuya@cyberagent.co.jp

Abstract

Three-variable first-order logic (FO^3) and the calculus of relations (CoR) define the same binary queries, an equivalence going back to Tarski in the 1940s. While the classical translation $\text{FO}^3 \Rightarrow \text{CoR}$ is exponential, we prove that this blow-up is unavoidable, resolving a long-standing open question. We construct positive formulas φ with a single quantifier whose equivalent terms require size $2^{\Omega(|\varphi|)}$, even over finite structures and circuit representations with subterm sharing. Our proof uses a preservation argument over a single finite structure. This approach applies beyond our primary question, establishing the lower bound even for size-specific circuits and bounded-error randomized circuits, and yielding an analogous exponential gap for the matrix query language MATLANG.

1 Introduction

Three-variable first-order logic (FO^3) over binary relations and the calculus of relations (CoR) define the same binary queries, an equivalence going back to Tarski’s 1940s work [52]. CoR is the algebra of binary relations under union $P \cup Q$, intersection $P \cap Q$, complement $\overline{P}$, converse $P^\smile = \{(y, x) : (x, y) \in P\}$, and the composition, or relative product, $P; Q = \{(x, y) : \exists z. (x, z) \in P, (z, y) \in Q\}$. CoR has no individual variables, so quantifier elimination has to go through the composition:

$$\exists z (P(x, z) \wedge Q(z, y)) \iff (P; Q)(x, y). \quad (1)$$

Using (1) together with bottom-up disjunctive normal form (DNF) conversion [27, 32, 49], FO^3 formulas can be translated into CoR with a single-exponential blow-up. Both were in place before Tarski took up the question (see Section 1.3). For an example, fix $k \geq 2$ and consider the FO^3 formula

$$\Psi_k(x, y) := \exists z \bigwedge_{i=1}^k (R_i(x, z) \vee R_i(z, y)), \quad (2)$$

which asks for a single witness z satisfying $R_i(x, z) \vee R_i(z, y)$ for every $i \in \{1, \dots, k\}$. To apply (1), the body under the quantifier must be factored into the form $\alpha(x, z) \wedge \beta(z, y)$. Thus, by DNF conversion, the body of (2) expands into 2^k disjuncts, one for each subset $I \subseteq [k]$, the disjunct for I asserting that $R_i(x, z)$ holds for $i \in I$ and $R_i(z, y)$ for $i \notin I$. Applying (1) to each disjunct gives an equivalent CoR term of size $2^{O(k)}$,

$$\text{CoR}_{\Psi_k} := \bigcup_{I \subseteq [k]} \left(\left(\bigcap_{i \in I} R_i \right); \left(\bigcap_{i \notin I} R_i \right) \right), \quad (3)$$

where an empty intersection denotes the full relation $\mathbf{1}$, which holds at every pair. For $k = 3$ the eight choices of I give eight disjuncts and eight compositions:

I	disjunct of the DNF	term
$\{1, 2, 3\}$	$R_1(x, z) \wedge R_2(x, z) \wedge R_3(x, z)$	$(R_1 \cap R_2 \cap R_3); \mathbf{1}$
$\{1, 2\}$	$R_1(x, z) \wedge R_2(x, z) \wedge R_3(z, y)$	$(R_1 \cap R_2); R_3$
$\{1, 3\}$	$R_1(x, z) \wedge R_3(x, z) \wedge R_2(z, y)$	$(R_1 \cap R_3); R_2$
$\vdots$	$\vdots$	$\vdots$
$\emptyset$	$R_1(z, y) \wedge R_2(z, y) \wedge R_3(z, y)$	$\mathbf{1}; (R_1 \cap R_2 \cap R_3)$

That the translation is exponential in general is classical.

Proposition 1.1 (well known; see Section 1.3). *Every FO^3 formula φ over binary relations with at most two free variables is equivalent, on all structures, to a CoR term of size $2^{O(|\varphi|)}$, where $|\cdot|$ counts the nodes of a syntax tree.*

Whether this single-exponential upper bound is optimal has remained a long-standing open question, and no matching lower bound for full CoR has been known since the correspondence itself was established.

In this paper we prove that the blow-up is unavoidable: the $2^{\Omega(|\varphi|)}$ lower bound holds already over finite structures, already for positive FO^3 formulas with a single quantifier, and even when the CoR side is a circuit rather than a term.

1.1 Main results

Write $|\varphi|$ and $|t|$ for the number of nodes in the syntax tree of a formula φ or a term t , and $\#_;(t)$ for the number of *composition* nodes of t ; in the second measure the Boolean operations and converse are free. A *relational circuit* $\mathcal{C}$ is a term whose subterms may be shared, that is, a directed acyclic graph, and $g_:(\mathcal{C})$ counts its distinct composition gates. Write $t \equiv_{\text{fin}} \varphi$ if t and φ define the same relation on every finite structure, and likewise for circuits. The first hard query is Ψ_k itself, and the 2^k compositions of (3) are close to necessary.

Theorem 1.2 (Disjunctive clauses). *Let $k \geq 2$. Every CoR term $t \equiv_{\text{fin}} \Psi_k$ satisfies*

$$\#_;(t) \geq \left\lceil \frac{1}{2} \binom{k}{\lfloor k/2 \rfloor} \right\rceil \quad \text{and} \quad |t| \geq \binom{k}{\lfloor k/2 \rfloor} + 1,$$

and the bound on $\#_;(t)$ holds more generally for circuits: every relational circuit $\mathcal{C} \equiv_{\text{fin}} \Psi_k$ has $g_:(\mathcal{C}) \geq \left\lceil \frac{1}{2} \binom{k}{\lfloor k/2 \rfloor} \right\rceil$.

Since $\binom{k}{\lfloor k/2 \rfloor} = \Theta(2^k / \sqrt{k})$, this falls short of the 2^k compositions of (3) by a factor of $\Theta(\sqrt{k})$. The second hard query closes the gap. It asks the two pairs (x, z) and (z, y) to satisfy the same one of two symbols in each clause, rather than to share out the k clauses between them. For $k \geq 1$ let

$$\Phi_k(x, y) := \exists z \bigwedge_{i=1}^k \left((P_i(x, z) \wedge P_i(z, y)) \vee (Q_i(x, z) \wedge Q_i(z, y)) \right), \quad (4)$$

over the $2k$ independent symbols $P_1, Q_1, \dots, P_k, Q_k$. Clause i holds at a witness z when both edges (x, z) and (z, y) lie in P_i , or both lie in Q_i .

Theorem 1.3 (Profile agreement). *Let $k \geq 1$. Every CoR term $t \equiv_{\text{fin}} \Phi_k$ satisfies $\#_{\cdot}(t) \geq 2^{k-1}$ and $|t| \geq 2^k + 1$, and the bound on $\#_{\cdot}(t)$ holds more generally for circuits: every relational circuit $\mathcal{C} \equiv_{\text{fin}} \Phi_k$ has $g_{\cdot}(\mathcal{C}) \geq 2^{k-1}$. The explicit term CoR_{Φ_k} of (17) uses 2^k compositions, so the composition count is determined within a factor of two.*

Both formulas are positive, use a single quantifier, and have size $\Theta(k)$ over a vocabulary that grows with k (Remark 4.1), so the two theorems answer the question above.

Corollary 1.4 (Succinctness gap). *The translation of Proposition 1.1 is optimal up to the constant in the exponent: there are positive FO^3 formulas φ with one quantifier, of arbitrarily large size, such that every relational circuit equivalent to φ over finite structures has size $2^{\Omega(|\varphi|)}$.*

Proof. Take $\varphi = \Psi_k$, of size $\Theta(k)$. By Theorem 1.2 a circuit for it has at least $\frac{1}{2} \binom{k}{\lfloor k/2 \rfloor}$ composition gates, and $\binom{k}{\lfloor k/2 \rfloor} \geq 2^k / (k+1)$, so its size is at least $2^{k-O(\log k)} = 2^{\Omega(|\varphi|)}$, while (3) is an equivalent term of size $O(k2^k) = 2^{O(|\varphi|)}$. Taking $\varphi = \Phi_k$ gives 2^k in place of $2^k / (k+1)$. $\square$

Section 1.4 outlines the proof of Theorem 1.2, which works also for Theorem 1.3.

What Corollary 1.4 leaves open is the same question over a vocabulary *fixed* in advance. The formulas above use k and $2k$ relation symbols respectively, and $|\varphi| = \Theta(k)$ because the size charges an occurrence of a symbol one node; whether an exponential gap holds for a family over a fixed signature we do not know (Remark 4.1).

1.2 Additional results

Size-specific and randomized targets. A CoR term is a single expression, evaluated on structures of every size, so Theorems 1.2 and 1.3 are uniform lower bounds. The same bounds hold non-uniformly, with the term or circuit chosen for each domain size n , and a bounded probability of error costs only a constant factor.

Theorem 1.5 (informal; see Theorem 5.4 and Corollary 5.5). *Let φ be Ψ_k or Φ_k . For every n above a threshold $n_0 = 2^{O(k)}$, the lower bounds of Theorems 1.2 and 1.3 hold for every term or circuit that agrees with φ on all structures with exactly n elements, even when that term is chosen depending on n . A randomized circuit that is correct with probability at least $1 - \varepsilon$ on every input, for some $\varepsilon < 1/2$, still needs a $(1 - 2\varepsilon)$ fraction of that same number of compositions, in expectation.*

Matrix query languages. The composition $P; Q$ can be replaced by $\bigvee_z P(x, z) \odot Q(z, y)$, with $\odot$ an arbitrary binary operation on the scalars and $\bigvee$ an arbitrary idempotent join (Section 3.3), and the lower bounds continue to hold; the min-plus matrix product $\min_z (P(x, z) + Q(z, y))$ is one instance. Brijder et al. [7] showed that MATLANG defines the same binary queries as a positive relational algebra on semiring-annotated relations whose intermediate results have at most three attributes; their translation into MATLANG is exponential, and they asked whether that blow-up is unavoidable.

Theorem 1.6 (informal; see Theorem 6.4, and Corollary 6.5 for non-idempotent semirings). *Fix the min-plus semiring, or max-plus, or max-min. For every k there is a positive relational algebra expression E_k of size $O(k)$, with binary output, whose intermediate results have at most three attributes. Every MATLANG computation equivalent to E_k on $n \times n$ inputs uses at least $2^k/3$ matrix*

products contracting the dimension n , even with arbitrary uniform entrywise functions and shared intermediates; 2^k products suffice.

1.3 Related work

The expressiveness question. For historical background, we refer to the surveys by Pratt [47], Maddux [36], and Burris and Legris [9], the monograph of Brady [4], and the historical notes of Tarski and Givant [52]. CoR is older than first-order logic. De Morgan introduced composition and converse in his papers on the syllogism of 1850 and 1860 [40, 41], Peirce developed the algebra through the 1870s and 1880s [44, 45], and Schröder devoted an entire volume to it in 1895 [50]. In contrast, first-order logic was formalized only in Hilbert’s lectures of 1917–18 and in Hilbert and Ackermann’s 1928 book [15, 27, 39]. In the terminology of the time, to *condense* a quantified statement about relations meant eliminating its quantifiers to express it within the calculus [35, p. 233]—the translation of Proposition 1.1 under its older name. Schröder claimed in 1895 that every such statement could be condensed [50, p. 551]. Korselt found a counterexample, published by Löwenheim in 1915 [35], which lies exactly on the variable boundary studied here: “the domain has at most two elements” requires three variables (FO^3) and can be condensed, whereas “the domain has at most three elements” requires four (FO^4) and cannot. Statements in FO^4 were thus known to lie outside CoR before first-order logic existed as a formal system, while the status of FO^3 remained open. Tarski’s 1941 paper presented further FO^4 statements outside CoR, observed that the calculus remained at essentially the same stage of development as forty-five years earlier, and posed the problem of characterizing the condensable sentences [51, pp. 74, 89]. According to Tarski and Givant, Tarski obtained the FO^3 characterization in the early 1940s and presented it at a 1945 Berkeley seminar. It was first published with full proofs in their 1987 monograph [52, pp. xvii, 88–89], and Maddux provided a modern treatment in his monograph [37]. This settled the expressiveness question and forms the basis for Proposition 1.1 and Corollary 1.4, but the computational cost of this translation was not examined.

The succinctness question. Expressive equivalence, however, implies nothing about formula size. Tarski and Givant defined their translation recursively, providing no bound on its growth [52, §3.9]. The folklore translation behind Proposition 1.1 applies bottom-up DNF conversion to the quantifier-free Boolean shell of a formula—the propositional normal-form theorem of Hilbert and Ackermann [27], read as a conversion procedure by Kleene [32] and Schönning [49]—and then applies (1) to each disjunct. This bottom-up conversion is the sole source of the exponential blow-up. In contrast, the reverse translation is linear: structural induction turns a CoR term of size m into an FO^3 formula of size $O(m)$, so the blow-up occurs in one direction only. Whether this exponential blow-up is unavoidable has remained open since the equivalence was established. The query Ψ_k is due to Nakamura [42], who showed that it has no subexponential translation into *positive* CoR (the fragment without complement), leaving the case of full CoR open. Theorems 1.2 and 1.3 settle this question.

Succinctness of logics and lower-bound methods. Succinctness is a standard measure for comparing logics of the same expressive power [22], and gaps of every size are known between fragments of first-order, temporal, and modal logics [1, 13, 14, 17]. The closest result to ours concerns finite-variable fragments over a single binary relation: on finite linear orders, FO^4 is exponentially

more succinct than FO^3 , while two- and three-variable logics are polynomially equivalent [22]. Such lower bounds are proved using the formula-size games of Adler and Immerman [1, 24] or, equivalently, by tracking a potential function along syntax trees [22]. This framework is indifferent to negation (which merely swaps structure pairs) and charges a bounded potential increase for each quantifier (corresponding to composition in CoR). The core difficulty lies in finding suitable structures and a potential, neither of which transfers from linear orders to our setting, for two concrete reasons.

First, on linear orders there is no gap to find: by the lower-bound theorem of Grohe and Schweikardt [22], an FO^3 formula of size s can only tell in which order its points come and whether two of them lie at distance exactly d for some $d \leq 4s^2$, and each such test is a CoR term of size $O(d)$, namely a d -fold composition of $\text{succ} = (<) \cap (\leq; <)$. As a formula of size s is a Boolean combination of the $s^{O(1)}$ available tests, it has an equivalent CoR term of size $s^{O(1)}$; so FO^3 and CoR terms are polynomially equivalent there.

Second, the mechanism of their gap is fundamentally different. A fourth variable permits the divide-and-conquer recursion familiar from Savitch’s theorem [48],

$$\varphi_m(x, y) = \exists z \forall u ((u = x \vee u = y) \rightarrow \varphi_{m-1}(z, u)),$$

which expresses “ x and y are at distance 2^m ” in size $O(m)$ by using one copy of φ_{m-1} for both halves of the path [22]; with three variables the recursion must write φ_{m-1} twice at every level, and their lower bound shows that no other FO^3 formula does better than $2^{\Omega(m)}$. Reuse is exactly what a circuit provides: a relational circuit expresses distance 2^m with m compositions, by squaring $S_{i+1} = S_i; S_i$ from $S_0 = \text{succ}$, whereas a term needs size $2^{\Omega(m)}$ by their bound and the linear translation of terms into FO^3 . Our bounds hold for circuits—where on linear orders the gap actually runs in the opposite direction—showing that the gap between FO^3 and CoR is not simply one of reuse.

This gap stems from how the two formalisms handle witnesses: in Ψ_k , the quantifier binds $R_i(x, z) \vee R_i(z, y)$ for all i simultaneously, constraining both edges at once. In contrast, a composition $P; Q$ splits the edges into P and Q . The classical translation (3) must therefore spend one composition on each way of partitioning the k clauses between the two edges, an overhead that Theorem 1.2 proves to be unavoidable. Nakamura [42] proved the exponential bound for positive CoR using a formula-size game; however, the argument relies fundamentally on positivity and does not extend to complement, which exchanges the two sides and thus requires a symmetric potential. Our preservation theorem supplies one: for a single finite structure and the family of substructures obtained by deleting one part each, the potential of a subterm is a set of parts outside of which every deletion is invisible to it. Boolean operations and converse contribute nothing to this set, while a composition contributes at most two parts; because these parts combine by union rather than by addition, a shared gate is charged only once, directly yielding the bound for circuits. Finally, Ψ_k and Φ_k , suitably interpreted, detect every deletion.

Composition as matrix multiplication. Since composition corresponds to a Boolean matrix product, a CoR term with m compositions evaluates a query using m fast matrix multiplications in time $O(mn^\omega)$, rather than the $O(n^3)$ time of naive enumeration, where ω is the matrix multiplication exponent. Bounded-variable queries can be evaluated in time polynomial in both the query and the structure [54]. Williams [55] exploited this speedup for three-quantifier graph properties, leading to a fine-grained complexity theory of first-order queries parameterized by variable count and quantifier structure [8, 18, 19]. The composition counts in Theorems 1.2 and 1.3 therefore lower-bound the

number of matrix products required by any relational evaluation of the query, a bound stated directly in Theorem 6.4. These results are not lower bounds for general Boolean circuits with access to individual input entries. In our cost measure, all Boolean operations (including complement) are free. In contrast, classical monotone lower bounds for a single Boolean matrix product [31, 38, 43, 46] and exact bounds for straight-line programs over semirings [28] count individual gates; see Jukna [29] for a survey.

Relational and matrix query languages. Algebraic representations of logical queries have been central to query processing since Codd [11, 12]. Binary relation algebra and its fragments form the algebraic core of graph and navigational query languages [16, 25, 26]. In these settings, logic-to-algebra translations can incur exponential blowups [2]. The succinctness of relational representations is therefore a central concern [16], because the size of the translated expression directly dictates the cost of query optimization and evaluation. MATLANG is a matrix query language built from matrix multiplication, transpose, ones-vectors, diagonalization, and pointwise functions [5]; see Geerts et al. [20] for a survey. For relations annotated with values in a semiring, in the sense of provenance semirings [21], Brijder et al. [7] proved that binary-output ARA(3)—the three-attribute annotated relational algebra—is expressively equivalent to MATLANG, mirroring the equivalence between FO^3 and CoR. An earlier manuscript asked whether this exponential arity elimination (and by analogy, the classical FO^3 translation) is unavoidable [6, p. 9]. Theorem 6.4 resolves this question affirmatively for commutative idempotent semirings of finite join breadth, while Section 6.4 extends the bound to additional semirings under a restricted operation set.

1.4 Technical overview

We outline the proof of Theorem 1.2. Theorem 1.3 uses the same construction and the same preservation theorem, and differs only in the interpretation of the relation symbols.

1.4.1 Outline

The term (3) contains one composition for each way of distributing the k clauses between the two edges of the witness. A more economical term might use complementation, factor out common subterms, or share them, and a normal-form argument would have to account for all of these possibilities. We therefore do not analyze the syntax of a candidate term at all. Instead, we fix a single finite structure, delete parts of it, and ask which of the resulting substructures a given term can still distinguish from the original.

We say that a term or a query *distinguishes* two structures over the same vocabulary, one of them a substructure of the other, if the relations that it defines on them differ at some pair of vertices of the smaller structure; equivalently, the term or query *detects* the deletion that produced the smaller structure. The proof has three steps.

- (i) We build one finite structure $\mathfrak{M}$ (Figure 1). Apart from a hub and some anchors, its vertices are partitioned into *cells* C_λ , one for each color λ in a finite set Λ of at least two colors. The only substructures we use are those obtained by deleting entire cells: for nonempty $J \subseteq \Lambda$, let $\mathfrak{M} \setminus J$ be $\mathfrak{M}$ with every cell C_λ such that $\lambda \notin J$ removed. Thus no vertex is ever removed individually, and the hub and the anchors survive every deletion.
- (ii) To a CoR term t with m compositions we attach a set $K \subseteq \Lambda$ of at most $2m$ colors, and show

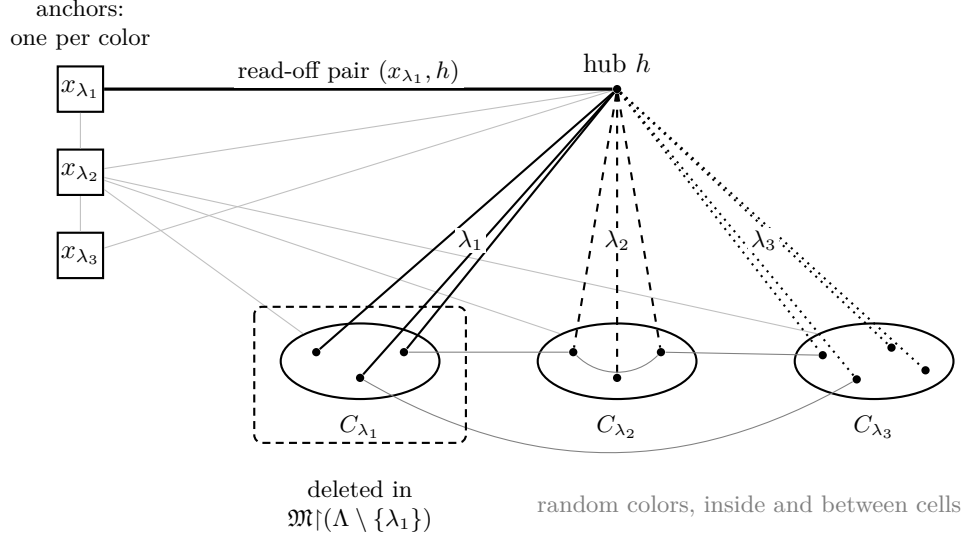


Figure 1: The structure $\mathfrak{M}$ with three colors. Each ellipse is a cell and the dots inside it are cell vertices; all cells have the same (large) number of vertices, and the number drawn here is not significant. Every pair of *distinct* vertices is an edge; only a few are drawn. The hub is joined to each vertex of C_λ by an edge of color λ (solid, dashed and dotted for $\lambda_1, \lambda_2, \lambda_3$), so the hub alone can distinguish the cells. Edges between cell vertices, inside a cell or across cells, carry random colors (gray) and realize every color between every pair of cells. Anchors, one per color, are joined to all vertices and their edges carry no color (light gray); they are distinguished by the relation symbols and are never deleted. Deleting the cell C_{λ_1} removes all of its vertices but leaves the read-off pair (x_{λ_1}, h) in place.

that t does not distinguish $\mathfrak{M}$ from $\mathfrak{M} \upharpoonright J$ for any nonempty $J \supseteq K$ (Theorem 3.7): deleting cells is invisible to t as long as every cell whose color lies in K is retained.

- (iii) We interpret the relation symbols of Ψ_k on $\mathfrak{M}$ so that Ψ_k does distinguish $\mathfrak{M}$ from $\mathfrak{M} \upharpoonright (\Lambda \setminus \{\lambda\})$, for every single color λ (Section 4).

Suppose $t \equiv_{\text{fin}} \Psi_k$, and let λ be any color. By (iii) the formula distinguishes $\mathfrak{M}$ from $\mathfrak{M} \upharpoonright (\Lambda \setminus \{\lambda\})$, and hence so does t . Since $|\Lambda| \geq 2$, the set $\Lambda \setminus \{\lambda\}$ is nonempty, so by (ii) this is possible only if $\Lambda \setminus \{\lambda\} \not\supseteq K$, that is, only if $\lambda \in K$. Hence $K = \Lambda$ and $2m \geq |\Lambda|$. Taking for Λ the $\lfloor k/2 \rfloor$ -element subsets of $[k]$ gives the composition bound of Theorem 1.2; the bound on $|t|$ follows from (5).

The remainder of this section treats the three steps in order. Section 1.4.2 constructs $\mathfrak{M}$ and specifies the pairs of vertices at which a query is read off; Sections 1.4.3 to 1.4.5 prove (ii) by showing that what a term computes on $\mathfrak{M}$ is too coarse to detect an individual cell; and Section 1.4.6 establishes (iii).

1.4.2 The structure (Section 3.1)

Recall that Λ is a finite set of *colors*. The vertices of $\mathfrak{M}$ are of three kinds, each with a distinct role (Figure 1):

	Number	Role
hub h	one	the only vertex that can distinguish the cells
cells C_λ	one per color $\lambda \in \Lambda$	contain the witnesses; a deletion removes a whole cell
anchors x_λ	one per color, never deleted	the query is read off at the pair (x_λ, h)

Every pair of distinct vertices is an *edge*, and every edge not incident to an anchor carries a color $\chi(u, v) = \chi(v, u) \in \Lambda$. The hub's edges announce the cell,

$$\chi(h, u) = \lambda \quad \text{for } u \in C_\lambda,$$

while the edges between two *cell vertices*, inside one cell or across two, are colored at random. Lemma 3.1 then yields the following *extension property*:

for all *distinct* cell vertices u, v , all colors μ, ν , and every cell C_λ , some $z \in C_\lambda$ other than u and v satisfies $\chi(u, z) = \mu$ and $\chi(z, v) = \nu$.

Thus a cell vertex sees every color in every cell and cannot distinguish one cell from another, whereas the hub can: it sees each cell in a single color. Edges incident to an anchor are left uncolored; anchors are instead distinguished by the interpretation of the relation symbols (Section 4).

Why anchors are needed. We call a pair (x_λ, h) a *read-off pair*. Such a pair must serve two purposes, one supplied by each of its two vertices.

First, it must survive every deletion. The test for a color λ compares $\mathfrak{M}$ with $\mathfrak{M} \upharpoonright (\Lambda \setminus \{\lambda\})$, so the query must be read off at a pair that is present in both structures, and Section 4 assigns a different pair to each color. A cell vertex does not survive the deletion of its own cell, and the hub is a single vertex, so the construction needs $|\Lambda|$ further vertices that are never deleted and that the relation symbols can distinguish. These are the anchors. They are indispensable rather than merely convenient: both hard formulas have the form $\exists z \psi$ in which every atom of ψ relates z to x or to y , and for formulas of this form Lemma B.1 in Appendix B bounds the number of detectable colors by $2|X| + 2$, where X is the set of anchors, for every interpretation of the kind used in this paper (Section 1.4.3). The number of anchors is thus precisely the resource that the lower bound consumes.

Second, it must be able to see the cell of a witness. The edge from a witness $z \in C_\lambda$ to the hub has color λ , whereas the edges from a cell vertex into C_λ realize every color, by the extension property. Hence the hub is the only vertex from which the cell of a witness can be read off.

An anchor, by contrast, has no colored edges at all. The only color associated with x_λ is the λ in its name; we call it the *label* of the anchor, and the interpretation of the relation symbols in Section 4 will make it readable. Labels and colors are the same objects; the two words merely indicate whether the object is carried by an anchor or by an edge. Consequently, a query can compare a witness against a label only at a read-off pair (x_λ, h) : the anchor supplies the label, the hub supplies the color of the witness, and the query tests whether the two agree.

This completes step (i) of the outline. The read-off pairs play no further role until Section 1.4.6, where Ψ_k performs exactly the comparison just described and (13) records the result; step (ii), proved in Sections 1.4.3 to 1.4.5, concerns all pairs of $\mathfrak{M}$ simultaneously, and only step (iii) uses one designated pair per color.

1.4.3 Types and type-constant relations (Section 3.2)

Pairs of vertices are sorted into finitely many *types*, each of which records only part of the information that the construction attaches to a pair.

Concretely, the type of an ordered pair (u, v) is

the pair (u, v) itself	if u and v are both anchors,
<i>out of</i> x	if u is the anchor x and v is not an anchor,
<i>into</i> x	if v is the anchor x and u is not an anchor,
<i>loop</i>	if $u = v$ and u is not an anchor,
<i>color</i> λ	if $u \neq v$, neither is an anchor and $\chi(u, v) = \lambda$.

We write $\text{tp}(u, v)$ for this type, and τ_λ for the type *color* λ ; the formal definition, with names for the remaining types, is (7). The number of types is finite and independent of the size of the cells: there is one type per ordered pair of anchors, one *out of* and one *into* type per anchor, one type for the loops, and one type per color.

A type records which anchors are involved, whether the two endpoints coincide, and the color of the edge between them. It does not record which cell an endpoint lies in, which vertex of that cell it is, or whether the hub is an endpoint at all: in Figure 1, an edge inside C_{λ_2} , an edge from C_{λ_1} to C_{λ_3} , and an edge from the hub into C_{λ_1} all have the same type whenever they carry the same color. Types are thus a deliberate coarsening rather than the finest available classification of pairs. In particular, although the hub is clearly visible in $\mathfrak{M}$, what makes it special is not that types can see it but that the colors of its edges are deterministic (Section 1.4.4).

Call a binary relation P on the vertices *type-constant* if $P(u, v)$ depends only on $\text{tp}(u, v)$, and write $P(\tau)$ for the common value of P on the pairs of a type τ . Not every relation is type-constant. Consider, for example, the relation $P = C_{\lambda_1} \times C_{\lambda_1}$, which holds at (u, v) exactly when both endpoints lie in the first cell of Figure 1. It holds at the loop (u, u) of a vertex $u \in C_{\lambda_1}$ but fails at the loop (h, h) of the hub, although these two pairs have the same type, namely *loop*; hence P is not constant on that type. The relation symbols of Ψ_k and Φ_k will be interpreted by type-constant relations (Section 4), and type-constancy then propagates to everything computed from them.

Everything computed is type-constant. Let P, Q be type-constant. In

$$(P; Q)(u, v) = \bigvee_z P(u, z) \wedge Q(z, v), \quad \text{the join taken over all vertices } z,$$

the term that a witness z contributes depends only on the pair of types $(\text{tp}(u, z), \text{tp}(z, v))$, and on nothing else about z . We call this pair the *profile* of z at (u, v) . The value of the composition is therefore determined by the *set* of profiles that some witness realizes: not by how many witnesses realize a profile, and not by which vertices they are.

Lemma 3.2 states that this set of realized profiles depends only on the type of (u, v) . If (u, v) is an edge of color λ , for instance, the realized profiles are (color μ , color ν) for every pair of colors, by the extension property; (loop, color λ) and (color λ , loop), realized by $z = u$ and $z = v$; and (into x , out of x) for each anchor x . Hence $P; Q$ is again type-constant. Boolean operations and converse trivially preserve type-constancy (Corollary 3.3), so every relation that a CoR term computes on $\mathfrak{M}$ from type-constant inputs is a table with one entry per type. In particular, no term can single out a cell, let alone an individual vertex.

1.4.4 A composition sees at most two colors (Section 3.3)

Fix two type-constant relations P, Q and abbreviate their values on the edges of color λ as

$$p_\lambda = P(\tau_\lambda), \quad q_\lambda = Q(\tau_\lambda).$$

As in step (i) of the outline, delete the cells of all colors outside a nonempty set $J \subseteq \Lambda$. A deletion removes witnesses, and a witness matters only through its profile, so the deletion can change the value of $P; Q$ at a pair only if it removes the *last* witness of some profile at that pair. At a pair with no hub endpoint this never happens: by the extension property, a single surviving cell already realizes every profile that the deleted cells realized. At a pair with a hub endpoint it can happen, but only in a limited way. Every edge from the hub into C_λ has color λ , so the hub sees a witness $z \in C_\lambda$ only through p_λ and q_λ : $P(h, z) = p_\lambda$ and $Q(z, h) = q_\lambda$. Consider the contribution of such a z to the join $(P; Q)(u, v) = \bigvee_z P(u, z) \wedge Q(z, v)$ at a pair (u, v) that has the hub as an endpoint. The hub may be the left endpoint, the right endpoint, or both, and in these three cases the contribution of z is

$$\begin{aligned} p_\lambda \wedge Q(\text{tp}(z, v)) & \quad \text{when } u = h \text{ and } v \neq h, \\ P(\text{tp}(u, z)) \wedge q_\lambda & \quad \text{when } v = h \text{ and } u \neq h, \\ p_\lambda \wedge q_\lambda & \quad \text{when } u = v = h. \end{aligned}$$

At a pair of $\mathfrak{M} \upharpoonright J$, the anchors and the two endpoints themselves survive the deletion and contribute the same terms regardless of J . The only part of the join that J can affect is therefore the part ranging over the cell witnesses, that is, over the colors $\lambda \in J$ and, for each such color, over the vertices of C_λ . In the first case, as z ranges over a cell, the factor $Q(\text{tp}(z, v))$ takes the same set of values whichever cell is chosen, by the extension property; the join therefore factors as $(\bigvee_{\lambda \in J} p_\lambda) \wedge c$ with c independent of J , and the only information about J that survives is whether some $\lambda \in J$ has $p_\lambda = 1$. The second case is symmetric. In the third case both edges of z carry the color λ , and what survives is whether some $\lambda \in J$ has $p_\lambda = q_\lambda = 1$. Thus J can influence the value only through three bits (Lemma 3.4):

whether some $\lambda \in J$ has $p_\lambda = 1$; whether some has $q_\lambda = 1$; whether some has both.

Two colors always suffice to fix all three bits (Lemma 3.6). If some color has $p_\lambda = q_\lambda = 1$, that color alone fixes all three; otherwise the third bit is 0, and one color for each of the first two bits suffices. We call such a set of at most two colors a *support* of the composition. As long as no cell whose color lies in the support is deleted, the value of the composition on the surviving vertices is unchanged.

For example, Figure 1 has three colors, so a family $(p_\lambda, q_\lambda)_{\lambda \in \Lambda}$ consists of two triples. Suppose first that

$$(p_{\lambda_1}, p_{\lambda_2}, p_{\lambda_3}) = (1, 1, 0), \quad (q_{\lambda_1}, q_{\lambda_2}, q_{\lambda_3}) = (0, 1, 1),$$

that is, P holds on the edges of colors λ_1 and λ_2 , and Q holds on those of colors λ_2 and λ_3 . Since $p_{\lambda_2} = q_{\lambda_2} = 1$, the color λ_2 alone fixes all three bits, so $\{\lambda_2\}$ is a support, and both C_{λ_1} and C_{λ_3} can be deleted without affecting the composition. Suppose instead that

$$(p_{\lambda_1}, p_{\lambda_2}, p_{\lambda_3}) = (1, 0, 0), \quad (q_{\lambda_1}, q_{\lambda_2}, q_{\lambda_3}) = (0, 1, 0).$$

Now no color has both of its values equal to 1: keeping λ_1 alone would change the second bit and keeping λ_2 alone would change the first, so every support must contain both colors, and $\{\lambda_1, \lambda_2\}$ is one. Lemma 3.6 states that this is the worst case.

1.4.5 From one composition to a whole term (Section 3.4)

A support controls a single composition. For a term t with m compositions, let K be the union of the supports of all of its compositions, so that $|K| \leq 2m$, and fix any nonempty $J \supseteq K$. Induction along the evaluation order then shows that the value of every subterm of t on $\mathfrak{M} \upharpoonright J$ is the restriction of its value on $\mathfrak{M}$ (Theorem 3.7): the Boolean operations and converse commute with restriction, and at a composition the two operands already agree by induction, while J contains the support of that composition, so Section 1.4.4 applies. Hence t does not distinguish $\mathfrak{M}$ from $\mathfrak{M} \upharpoonright J$ for any nonempty $J \supseteq K$, which is step (ii) of the outline. A shared subterm is charged only once, so the same argument bounds the number of composition gates of a circuit.

At a composition gate, the two relations being compared are always related by an inclusion in one direction, for a trivial reason: deleting cells removes witnesses and never adds any, so a pair related by the composition on $\mathfrak{M} \upharpoonright J$ is related by it on $\mathfrak{M}$ as well. Writing D for the vertex set of $\mathfrak{M}$, D_J for that of $\mathfrak{M} \upharpoonright J$, and $P \upharpoonright D_J$ for the set of pairs of a relation P that have both endpoints in D_J ,

$$(P \upharpoonright D_J); (Q \upharpoonright D_J) \subseteq (P; Q) \upharpoonright D_J$$

holds for every J . What can fail is the reverse inclusion: a pair may be related on $\mathfrak{M}$ only through a witness that the deletion removed. Excluding this is precisely the role of a support, which is why Section 1.4.4 yields an *equality* and not merely this inclusion.

Complement is the operation for which the distinction between inclusion and equality matters. The issue is not that complement fails to commute with restriction: a subterm is complemented within the structure on which it is evaluated, relative to D_J^2 on $\mathfrak{M} \upharpoonright J$ and to D^2 on $\mathfrak{M}$, and $D_J^2 \setminus (P \upharpoonright D_J) = (D^2 \setminus P) \upharpoonright D_J$ holds exactly, for every relation P . The issue is that complement reverses inclusions rather than preserving them. If only the inclusion displayed above were available, the two sides for the complemented subterm would satisfy

$$D_J^2 \setminus [(P \upharpoonright D_J); (Q \upharpoonright D_J)] \supseteq D_J^2 \setminus [(P; Q) \upharpoonright D_J],$$

with the inclusion now pointing in the opposite direction, and a further complement would reverse it again. No single inclusion can therefore be maintained through a term whose complements occur at arbitrary depths. An equality, by contrast, has no direction to reverse: $A = B$ yields $D_J^2 \setminus A = D_J^2 \setminus B$, and union, intersection and converse preserve equalities equally well. This is why complement gates require no separate treatment, why no monotonicity is used anywhere in the argument, and why the target may be an arbitrary CoR term rather than a positive one.

1.4.6 The formula detects every deletion (Section 4)

It remains to interpret the R_i . Recall that Ψ_k has one quantifier, $\Psi_k(x, y) = \exists z \bigwedge_{i=1}^k (R_i(x, z) \vee R_i(z, y))$. Evaluated at a pair (u, v) of vertices, it asks for a *witness*: a vertex z such that every one of the k clauses holds, either through the edge (u, z) or through the edge (z, v) .

So far Λ has been an arbitrary finite set of colors. We now fix it: let Λ be the set of $\lfloor k/2 \rfloor$ -element subsets of $[k]$, and write A, B for colors in place of λ, μ . There is one anchor x_A for each color A .

We now define the relations R_i . For all colors A and B ,

$$\begin{aligned} R_i(x_A, u) &\text{ holds exactly when } i \in A, && \text{for every vertex } u \text{ that is not an anchor;} \\ R_i(u, v) &\text{ holds exactly when } i \notin B, && \text{for every two distinct non-anchor vertices} \\ &&& u, v \text{ whose edge has the color } B; \end{aligned}$$

and R_i is false on every remaining pair. This interpretation is type-constant, as required by Section 1.4.3: the pairs from x_A to a non-anchor vertex form a single type (edges at an anchor carry no color), and the pairs of color B form a single type, whether they run from the hub into C_B or between two cell vertices.

At the read-off pair (x_A, h) , a vertex z of C_B satisfies clause i if $i \in A$, through the edge (x_A, z) , or if $i \notin B$, through the edge (z, h) ; so it satisfies all k clauses exactly when $B \subseteq A$, and since A and B have the same size, exactly when $B = A$. The hub is not a witness: R_i is false on the loop (h, h) , so at $z = h$ clause i would need $i \in A$, and A contains only $\lfloor k/2 \rfloor$ of the k indices. Nor is an anchor x_B : R_i is false on the pair (x_A, x_B) , so clause i would need $i \in B$, and $|B| = \lfloor k/2 \rfloor$ as well. The witnesses at (x_A, h) are therefore exactly the vertices of the single cell C_A , and

$$\Psi_k \text{ holds at } (x_A, h) \text{ in } \mathfrak{M} \upharpoonright J \iff A \in J,$$

which is (13). Deleting C_A therefore changes the value of Ψ_k at a pair that survives the deletion, so every color is detectable, which completes step (iii) of the outline.

The choice of labels is forced by the method, in a sense that can be made precise. At a read-off pair, the formula compares the set announced by the anchor with the set carried by the witness's edge to the hub, and a color is detected precisely when exactly one color passes this comparison. Recording, for each of the two sides, the set of clauses that it leaves unsatisfied turns the comparison into a disjointness test between two subsets of $[k]$, and detection into a cross-intersecting condition on pairs of such subsets. Bollobás's set-pair inequality (Lemma B.2), which bounds the size of such a family of pairs of subsets of $[k]$ by $\binom{k}{\lfloor k/2 \rfloor}$, then bounds the number of detectable colors by the same quantity (Proposition B.3 in Appendix B), and the interpretation above attains this bound. This explains why Theorem 1.2 involves a central binomial coefficient rather than 2^k . The resulting gap of a factor $\Theta(\sqrt{k})$ against the 2^k compositions of (3) is not closed in this paper; by the same proposition, it is inherent to Ψ_k and to this construction, and not an artifact of the particular relations we chose. The formula Φ_k avoids this loss by requiring agreement rather than inclusion, which allows all 2^k subsets of $[k]$ to serve as labels (Section 4.2). Finally, the only property of conjunction used in the argument is the one isolated in Section 1.4.4, namely that two colors fix its three bits. Section 6 replaces conjunction by a semiring product, and the same argument goes through with three colors in place of two.

1.5 Organization

Section 2 fixes the conventions. The body then falls into two parts, matching the two groups of results above. Part I proves Theorems 1.2 and 1.3: Section 3 builds the structure $\mathfrak{M}$ and proves the preservation theorem, and Section 4 interprets the two formulas on $\mathfrak{M}$. Part II collects the extensions: Section 5 passes to size-specific and randomized targets, and Section 6 treats semiring products and MATLANG. The two can be read in either order, except that the fixed-size statements of Section 6 rest on Proposition 5.3. Appendix A makes the coloring of $\mathfrak{M}$ explicit, Appendix B shows that the interpretations of Section 4 are optimal, and Appendix C collects remarks on other kernels and on wider scalars.

2 Preliminaries

Structures and formulas. A vocabulary is a finite set of binary relation symbols. A structure $\mathfrak{A}$ interprets each symbol R as a binary relation $R^{\mathfrak{A}}$ on a nonempty domain D ; all structures in this paper are finite. FO^3 is first-order logic with equality restricted to the three variable names x, y, z ; a name may be quantified repeatedly. For a formula φ whose free variables are among x, y , write $\varphi^{\mathfrak{A}} \subseteq D^2$ for the relation that it defines in $\mathfrak{A}$ and $\varphi^{\mathfrak{A}}(u, v) \in \{0, 1\}$ for its value at a pair. A formula with at most two free variables can be brought to this form by permuting the three names, which changes neither its size nor the relation it defines. The size $|\varphi|$ is the number of nodes of the syntax tree, so that $|\Psi_k| = \Theta(k)$ and $|\Phi_k| = \Theta(k)$.

The calculus of relations. CoR terms are built from the relation symbols and the constants $\mathbf{0} = \emptyset$, $\mathbf{1} = D^2$, and $\mathbf{1}' = \Delta_D$ (the identity relation) by union, intersection, complement relative to D^2 , converse $P^\smile = \{(v, u) : (u, v) \in P\}$, and composition

$$(P; Q)(u, v) \iff \exists z \in D [P(u, z) \wedge Q(z, v)].$$

The endpoints and the witness need not be distinct. We write $t^{\mathfrak{A}}$ for the relation defined by t in $\mathfrak{A}$, $|t|$ for the number of nodes of the syntax tree of t , and $\#_;(t)$ for its number of composition nodes. In a syntax tree all of whose nodes have arity at most two, the leaves outnumber the binary nodes by exactly one; since the compositions are among the binary nodes, a term has at least $\#_;(t) + 1$ leaves, and counting the leaves and the compositions alone already gives

$$|t| \geq 2\#_;(t) + 1. \quad (5)$$

A *relational circuit* $\mathcal{C}$ is a directed acyclic graph whose gates carry the same operations, whose sources are relation symbols and constants, and which has one designated output gate. It is evaluated in topological order, and an intermediate relation may be used by several gates. We write $g_+(\mathcal{C})$ for the number of composition gates and $|\mathcal{C}|$ for the *size*, the number of nodes of the graph, sources and gates together; this is the analogue for circuits of the node count $|t|$ of a term. Neither the depth nor the number of gates of other kinds is bounded, so the only relation between the two measures is $|\mathcal{C}| \geq g_+(\mathcal{C})$. A term is a circuit in which every gate is used once, so every statement about circuits below applies to terms.

A term or circuit is fixed once and for all and is evaluated on every finite structure over its vocabulary. It contains no constants beyond $\mathbf{0}$, $\mathbf{1}$ and $\mathbf{1}'$, which are interpreted in each structure; in particular, it contains no constant chosen for a particular domain or domain size. Section 5 shows that the lower bounds persist even when the circuit may depend on the domain size. We write $t \equiv_{\text{fin}} \varphi$ if $t^{\mathfrak{A}} = \varphi^{\mathfrak{A}}$ for every finite structure $\mathfrak{A}$, and use the same notation for circuits.

Notation. $[k] = \{1, \dots, k\}$, $2^{[k]}$ is the set of all subsets of $[k]$ and $\binom{[k]}{r}$ the set of its r -element subsets. Blackboard bold is reserved for number systems and for scalars: $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{R}$, $\mathbb{C}$, the field $\mathbb{F}_q$, the truth values $\mathbb{B}$, and $\mathbb{S}$ for a semiring; these never clash with the italic N for a number of colors or with $\mathcal{C}$ for a computation. From Section 3 on, u, v denote vertices of the finite structure, z a witness, h its hub and x one of its anchors. Colors are written λ, μ, ν where the set Λ is left abstract, and A or B where Λ is a concrete family of subsets of $[k]$; Section 3 uses only the first convention and Section 4 only the second, while Sections 5 and 6 use both. Cells are written C_λ , C_J , C_Λ , always with a

subscript, so that the unsubscripted $\mathcal{C}$ is unambiguously a circuit or a computation. The letters P, Q without a subscript are the two operands of an aggregation; P_i, Q_i with a subscript are the input symbols of (4). For $\emptyset \neq D' \subseteq D$, the substructure $\mathfrak{A} \upharpoonright D'$ has domain D' and relations $R^{\mathfrak{A}} \cap D'^2$, and for a relation P on D we write $P \upharpoonright D' = P \cap D'^2$. Every CoR operation commutes with this restriction except composition, whose witnesses are confined to D' : $(P; Q) \upharpoonright D' \supseteq (P \upharpoonright D'; Q \upharpoonright D')$, and the inclusion can be strict. Controlling this defect is the subject of the next section.

Part I: The main theorems

3 A preservation theorem for compositions

This section carries out steps (i) and (ii) of the outline in Section 1.4.1: it builds the structure $\mathfrak{M}$ and proves the preservation theorem (Theorem 3.7), for an arbitrary kernel in place of conjunction. The construction does not mention Ψ_k or Φ_k , whose interpretation is step (iii) (Section 4). The proof uses just one property of composition, isolated in Section 3.3, and this is what makes the theorem applicable to matrix products in Section 6.

The shape of the argument. The argument has three independent parameters, and each of the sections that follow varies one of them while holding the other two fixed: the interpretation of the source query on $\mathfrak{M}$ (Section 4, for Ψ_k and for Φ_k), the family of perturbations of $\mathfrak{M}$ (deletion of cells here, recoloring of the hub's edges on a fixed domain in Section 5), and the operation whose occurrences are counted (conjunction in Section 3.3, a semiring product and an arbitrary kernel in Section 6). A single number, the support number of Definition 3.5, couples the three, and it depends only on the operation and the join. Everything else is proved here for an arbitrary kernel, so Section 6 only has to recompute $\text{sn}(\odot)$; this is why the construction that separates FO^3 from CoR also answers the question of Brijder et al. [6] about MATLANG (Section 6.3).

Two features of the conclusion are worth recording, since they are not usually available together. First, the bound is proved against a fixed finite family of $|\Lambda| + 1$ structures, chosen in advance of the candidate and written down explicitly (Proposition A.1); no limit is taken and no infinite family is involved. Second, a shared subterm is charged only once, so the statement concerns circuits and not only terms. We claim no more than this: the invariant separates one structure from a bounded family of its perturbations, and the bound accordingly counts occurrences of a single operation in a language whose remaining operations are free. It is not a lower bound for Boolean circuits with access to individual entries of the input.

3.1 The structure

Fix a finite set Λ of *colors* with $N = |\Lambda| \geq 2$. The vertices of $\mathfrak{M}$ are of three kinds (Figure 1 on page 7):

- one *hub* h ;
- for each color $\lambda \in \Lambda$ a *cell* C_λ of $L = \lceil 12N^2 \ln(2N) \rceil$ *cell vertices*; the cells are disjoint, and $C_\Lambda = \bigcup_{\lambda \in \Lambda} C_\lambda$;
- a finite set X of *anchors*, one per color in Section 4 and possibly more in Section 5.

The *core* is $U = \{h\} \cup C_\Lambda$ and the domain is $D = X \cup U$. A pair of *distinct* vertices is called an *edge*. Every edge $\{u, v\}$ between two core vertices carries a color $\chi(u, v) = \chi(v, u) \in \Lambda$; edges incident to an anchor carry none. The hub's edges announce the cell,

$$\chi(h, u) = \lambda \quad \text{for } u \in C_\lambda, \quad (6)$$

and the edges between cell vertices are colored so that the following holds.

Lemma 3.1 (Extension property). *The edges between cell vertices can be colored so that*

- (a) *for all distinct $u, v \in C_\Lambda$, all colors $\mu, \nu \in \Lambda$, and every cell C_λ , some $z \in C_\lambda \setminus \{u, v\}$ satisfies $\chi(u, z) = \mu$ and $\chi(z, v) = \nu$;*
- (b) *for every $u \in C_\Lambda$, every color $\mu \in \Lambda$, and every cell C_λ , some $z \in C_\lambda \setminus \{u\}$ satisfies $\chi(u, z) = \mu$.*

Proof. Part (b) follows from (a) by choosing any $v \neq u$ and any ν . For (a), color the edges between cell vertices independently and uniformly at random. A *requirement* is a choice of λ , of distinct $u, v \in C_\Lambda$, and of μ, ν ; since $|C_\Lambda| = NL$ there are fewer than $N \cdot (NL)^2 \cdot N^2 = N^5 L^2$ of them. For a fixed requirement, each of the at least $L - 2$ vertices $z \in C_\lambda \setminus \{u, v\}$ satisfies it with probability N^{-2} , and these events are independent because they concern disjoint pairs of edges, so the requirement fails with probability at most $(1 - N^{-2})^{L-2} \leq \exp(-(L-2)/N^2)$. As $\ln(2N) \leq N$ and $N \geq 2$ we have $L \leq 13N^3$ and $(L-2)/N^2 \geq 12 \ln(2N) - \frac{1}{2}$, so the probability that some requirement fails is at most

$$N^5 L^2 \exp(-(L-2)/N^2) \leq 169 N^{11} \exp(1/2) (2N)^{-12} = \frac{169 \exp(1/2)}{4096 N} < 1.$$

Hence a coloring with property (a) exists. □

The constant 12 is sufficient rather than optimal: it is what produces the factor $1/N$ in the bound just displayed. Nothing below depends on its value, since L enters the results of this paper only through the size threshold (21).

We fix such a coloring once and for all. As explained in Section 1.4.2, only the hub distinguishes cells, by (6) and Lemma 3.1, and the anchors, which are never deleted, will be distinguished by the interpretation of the relation symbols. A pair (x, h) with x an anchor is called a *read-off pair*; these are the only pairs at which Section 4 evaluates the two hard queries.

The coloring of Lemma 3.1 is the only object in this paper obtained from a probabilistic existence argument, and this argument can be avoided: Appendix A gives a deterministic polynomial-time construction and a closed form over a prime field, in which the cells are the cosets of the N -th powers (Proposition A.1). Either construction makes the $|\Lambda| + 1$ structures of Corollary 3.8 explicit, and with the closed form the size threshold (21) remains polynomial in N . The cells cannot be made much smaller: property (a) of Lemma 3.1 already forces $L \geq N^2 + 1$ (Remark A.2).

3.2 Types, homogeneity, and profiles

Every ordered pair of vertices receives a *type*, by cases:

$$\text{tp}(u, v) = \begin{cases} \tau_{x,x'} & \text{if } u = x \in X \text{ and } v = x' \in X, \\ \tau_{x,*} & \text{if } u = x \in X \text{ and } v \in U, \\ \tau_{*,x} & \text{if } u \in U \text{ and } v = x \in X, \\ \tau_{=} & \text{if } u = v \in U, \\ \tau_{\chi(u,v)} & \text{if } u \neq v \text{ and } u, v \in U. \end{cases} \quad (7)$$

As sets of pairs, $\tau_{x,x'} = \{(x, x')\}$, $\tau_{x,*} = \{x\} \times U$, $\tau_{*,x} = U \times \{x\}$, $\tau_{=} = \Delta_U$, and $\tau_\lambda = \{(u, v) \in U^2 : u \neq v, \chi(u, v) = \lambda\}$; they partition D^2 . There are $|X|^2 + 2|X| + 1 + |\Lambda|$ types, however large the cells are.

Note that a cell is not a type, and neither is $\{h\}$: the pairs (h, u) with $u \in C_\lambda$ lie in τ_λ together with every other edge of color λ (Section 1.4.3).

An array $P : D^2 \rightarrow S$ with values in a set S — in particular a relation, where S is the set $\mathbb{B} = \{0, 1\}$ of truth values — is *type-constant* if it is constant on every type; we then write $P(\tau)$ for its value there. The point of this subsection is that type-constancy propagates to everything computed from type-constant inputs (Section 1.4.3), because a witness is seen only through the types of its two edges: for type-constant P, Q , the term $P(u, z) \wedge Q(z, v)$ that a witness z contributes to $(P; Q)(u, v)$ depends only on the *profile* $(\text{tp}(u, z), \text{tp}(z, v))$ of z at (u, v) , so the value of the composition is determined by the set of profiles that some witness realizes. The structure is built so that this set depends only on the type of the pair.

Lemma 3.2 (Homogeneity). *For every type τ , the set of witness profiles*

$$W_\tau = \{(\text{tp}(u, z), \text{tp}(z, v)) : z \in D\}$$

is the same for all pairs $(u, v) \in \tau$.

Proof. Let $(u, v) \in \tau$. A witness z is an anchor, one of the two endpoints, or another core vertex; we list the profiles of each kind and check that they do not depend on the choice of (u, v) in τ .

Case $\tau = \tau_\lambda$. Then $u \neq v$ are core vertices with $\chi(u, v) = \lambda$. An anchor $z = x$ gives $(\tau_{*,x}, \tau_{x,*})$; $z = u$ gives $(\tau_{=}, \tau_\lambda)$ and $z = v$ gives $(\tau_\lambda, \tau_{=})$. A core witness $z \notin \{u, v\}$ gives (τ_μ, τ_ν) with $\mu = \chi(u, z)$ and $\nu = \chi(z, v)$, and every pair (μ, ν) occurs: for $u, v \in C_\Lambda$ by Lemma 3.1(a); for $u = h$ choose $z \in C_\mu \setminus \{v\}$ with $\chi(z, v) = \nu$ by Lemma 3.1(b), so that $\chi(h, z) = \mu$ by (6); the case $v = h$ is symmetric.

Case $\tau = \tau_{=}$. Then $u = v \in U$. Anchors contribute as before, $z = u$ gives $(\tau_{=}, \tau_{=})$, and a core witness $z \neq u$ gives (τ_μ, τ_μ) with $\mu = \chi(u, z)$; every μ occurs, by Lemma 3.1(b) if $u \in C_\Lambda$ and by (6) if $u = h$.

Case $\tau = \tau_{x,}$.* Then $u = x$ is an anchor and $v \in U$. An anchor $z = x'$ (possibly $x' = x$) gives $(\tau_{x,x'}, \tau_{x',*})$; $z = v$ gives $(\tau_{x,*}, \tau_{=})$; a core witness $z \neq v$ gives $(\tau_{x,*}, \tau_\mu)$ with $\mu = \chi(z, v)$, and every μ occurs as in the previous case. The type $\tau_{*,x}$ is symmetric, and a type $\tau_{x,x'}$ consists of a single pair. $\square$

Homogeneity is exactly the property that composition needs.

Corollary 3.3 (Type-constant relations are closed). *Let P, Q be type-constant relations on D . Then $P; Q, P^\smile, \overline{P}, P \cup Q, P \cap Q$, and the constants $\mathbf{0}, \mathbf{1}, \mathbf{1}'$ are type-constant, and for every type τ*

$$(P; Q)(\tau) = \bigvee_{(\tau_1, \tau_2) \in W_\tau} P(\tau_1) \wedge Q(\tau_2). \quad (8)$$

Proof. For $(u, v) \in \tau$ we have $(P; Q)(u, v) = \bigvee_{z \in D} P(\text{tp}(u, z)) \wedge Q(\text{tp}(z, v))$, and by Lemma 3.2 the set of profiles over which the join ranges depends only on τ ; this is (8). Union, intersection, and complement act entrywise. Converse maps each type onto a type (τ_λ and τ_- onto themselves, $\tau_{x,*}$ onto $\tau_{*,x}$, $\tau_{x,x'}$ onto $\tau_{x',x}$). The constants $\mathbf{0}$ and $\mathbf{1}$ are constant, and $\mathbf{1}'$ is true exactly on the types τ_- and $\tau_{x,x}$. $\square$

Thus every relation computed on $\mathfrak{M}$ from type-constant inputs is a finite table with one entry per type.

3.3 Deleting cells: three responses

For a nonempty set $J \subseteq \Lambda$ let

$$D_J = X \cup \{h\} \cup C_J, \quad C_J = \bigcup_{\lambda \in J} C_\lambda,$$

and write $\mathfrak{M} \upharpoonright J$ and $P \upharpoonright J$ for $\mathfrak{M} \upharpoonright D_J$ and $P \upharpoonright D_J$: the cells of all colors outside J are deleted and everything else is retained. We ask how the value of a composition on $\mathfrak{M} \upharpoonright J$ depends on J .

Compositions as aggregations. It costs nothing to answer this question for a more general operation, which is what Section 6 needs. Let $(S, \vee, 0)$ be a join-semilattice with least element 0, so that finite joins are associative, commutative and idempotent, and let $\odot$ be any binary operation on S , called the *kernel*. For arrays $P, Q : D'^2 \rightarrow S$ on a domain D' , the *aggregation* of P and Q with kernel $\odot$ is

$$(P ;_\odot Q)(u, v) = \bigvee_{z \in D'} P(u, z) \odot Q(z, v). \quad (9)$$

For $S = \mathbb{B} = \{0, 1\}$, with disjunction as the join and $\odot = \wedge$, this is composition, $P ;_\wedge Q = P; Q$, and a reader interested only in Theorems 1.2 and 1.3 may read $p \odot q$ as $p \wedge q$ throughout. No algebraic law is assumed for $\odot$. A *computation* is a circuit whose gates are aggregations, possibly with different kernels at different gates, together with arbitrary functions $S^r \rightarrow S$ applied entry by entry, transpose, constant arrays, and the equality array, which takes two fixed distinct values on and off the diagonal. For $S = \mathbb{B}$ this includes every CoR term and every relational circuit. The gates other than aggregations are *free*: they commute with restriction to a subdomain, and they preserve type-constancy. Lemma 3.2 and (8) hold for every kernel, with $\odot$ in place of $\wedge$, since only the idempotence of $\vee$ was used in Corollary 3.3.

Where the retained colors enter. Let P, Q be type-constant arrays on D and write

$$p_\lambda = P(\tau_\lambda), \quad q_\lambda = Q(\tau_\lambda) \quad (\lambda \in \Lambda)$$

for their values on the edges of color λ . As explained in Section 1.4.4, whose argument does not depend on the kernel, a deletion can change the value of an aggregation only at a pair with a hub

endpoint, and the hub sees the whole of C_λ through p_λ and q_λ alone: a retained witness $z \in C_\lambda$ contributes a term $p_\lambda \odot s$, $s \odot q_\lambda$ or $p_\lambda \odot q_\lambda$, where s is the value carried by its other edge, according to whether the hub is the left endpoint, the right endpoint, or both. This motivates the three *responses* of the family $(p_\lambda, q_\lambda)_{\lambda \in \Lambda}$ to a set J of colors:

$$\ell_J(s) = \bigvee_{\lambda \in J} p_\lambda \odot s, \quad r_J(s) = \bigvee_{\lambda \in J} s \odot q_\lambda, \quad d_J = \bigvee_{\lambda \in J} p_\lambda \odot q_\lambda \quad (s \in S). \quad (10)$$

The *left* and *right responses* ℓ_J and r_J are functions on S ; the *paired response* d_J is a scalar. For $\odot = \wedge$ they amount to the three bits of Section 1.4.4.

These three responses are all that a deletion can disturb.

Lemma 3.4 (Three responses preserve an aggregation). *Let P, Q be type-constant arrays on D and $\emptyset \neq J \subseteq \Lambda$. If $(\ell_J, r_J, d_J) = (\ell_\Lambda, r_\Lambda, d_\Lambda)$, then*

$$(P ;_\odot Q) \upharpoonright J = (P \upharpoonright J) ;_\odot (Q \upharpoonright J),$$

where the right-hand side is evaluated on D_J .

Proof. Let $u, v \in D_J$. Both sides are joins over witnesses, the left one over $z \in D$ and the right one over $z \in D_J$. Witnesses in $X \cup \{h\}$ and witnesses in $\{u, v\}$ lie in D_J and contribute the same term to both sides. It remains to compare, for $I = J$ and for $I = \Lambda$, the contribution of the cell witnesses,

$$\sigma_I(u, v) = \bigvee_{z \in C_I \setminus \{u, v\}} P(u, z) \odot Q(z, v).$$

As P and Q are type-constant, $\sigma_I(u, v)$ is determined by the set of profiles realized by $z \in C_I \setminus \{u, v\}$. We read these off from (6) and Lemma 3.1 for each kind of endpoint. Following the rule $p_\lambda = P(\tau_\lambda)$, write $p_x = P(\tau_{x,*})$ and $q_{x'} = Q(\tau_{*,x'})$; the two subscript positions differ because P is always the left factor of the aggregation and Q the right one. The result is

$\sigma_I(u, v)$	$v = x' \in X$	$v = h$	$v \in C_J$
$u = x \in X$	$p_x \odot q_{x'}$	$r_I(p_x)$	$r_\Lambda(p_x)$
$u = h$	$\ell_I(q_{x'})$	d_I	$\bigvee_{\mu \in \Lambda} \ell_I(q_\mu)$
$u \in C_J$	$\ell_\Lambda(q_{x'})$	$\bigvee_{\mu \in \Lambda} r_I(p_\mu)$	$\begin{cases} d_\Lambda & \text{if } u = v, \\ \bigvee_{\mu, \nu \in \Lambda} p_\mu \odot q_\nu & \text{if } u \neq v. \end{cases}$

(11)

We verify the table row by row. For two anchor endpoints, every cell witness contributes the same term $p_x \odot q_{x'}$, so the entry does not depend on I . For an anchor x and the hub, a witness $z \in C_\lambda$ has $\text{tp}(z, h) = \tau_\lambda$, so the retained colors $\lambda \in I$ enter through r_I . For an anchor and a cell vertex v , the edges $\{z, v\}$ with z in any single retained cell already realize every color, by Lemma 3.1(b), so the entry does not depend on I . In the second row a witness $z \in C_\lambda$ is seen from the hub through p_λ : at (h, x') this gives $\ell_I(q_{x'})$; at (h, h) the two edges of z have the same color λ , which gives d_I ; at (h, v) with v a cell vertex the profiles are (τ_λ, τ_μ) with $\lambda \in I$ and μ arbitrary, by Lemma 3.1(b). The first two entries of the third row are obtained from the third column by exchanging the roles of

the two endpoints. Finally, for two cell-vertex endpoints, Lemma 3.1(a) yields every ordered pair of colors if $u \neq v$, while for $u = v$ both edges of z have the same color and every color occurs.

Only the entries containing ℓ_I , r_I or d_I depend on I , and the hypothesis makes them equal for $I = J$ and $I = \Lambda$; for instance $\bigvee_\mu \ell_J(q_\mu) = \bigvee_\mu \ell_\Lambda(q_\mu)$. Hence $\sigma_J(u, v) = \sigma_\Lambda(u, v)$ for all $u, v \in D_J$, and adding back the unchanged witnesses gives the identity. $\square$

The lemma applies to any type-constant operands, not only to inputs, and Lemma 3.2 makes it available at every gate of a computation. It says that an aggregation cannot distinguish J from Λ unless its three responses do; how many colors are needed to fix those responses is a property of the kernel alone.

Definition 3.5 (Support and support number). A set $K \subseteq \Lambda$ *supports* the family $(p_\lambda, q_\lambda)_{\lambda \in \Lambda}$ if $(\ell_K, r_K, d_K) = (\ell_\Lambda, r_\Lambda, d_\Lambda)$. The *support number* $\text{sn}(\odot)$ of a kernel $\odot$, with respect to the given join, is the least c such that every finite family, over every finite set of colors, has a support of size at most c ; it is ∞ if no such c exists. It therefore depends on the join as well as on $\odot$; the join is always the one fixed with S , and Section 6 varies both.

If K supports a family then so does every J with $K \subseteq J \subseteq \Lambda$, since $\ell_K \leq \ell_J \leq \ell_\Lambda$ pointwise and likewise for r and d . A support may be empty: as $\ell_\emptyset$ and $r_\emptyset$ are identically 0 and $d_\emptyset = 0$, the empty set supports exactly the families whose three responses to Λ vanish, which for $\odot = \wedge$ happens as soon as every p_λ and every q_λ is 0. For a general kernel that last inference is not available, since no algebraic law is assumed for $\odot$ and $0 \odot s$ need not be 0. Either way an empty support causes no trouble, since it is the retained set J of colors, never the support, that is required to be nonempty.

For the kernel that Theorems 1.2 and 1.3 need, two colors always suffice.

Lemma 3.6 (Conjunction has support number two). *For $S = \mathbb{B}$, with disjunction as the join and $\odot = \wedge$, one has $\text{sn}(\wedge) = 2$.*

Proof. Here $\ell_J(s) = s \wedge \bigvee_{\lambda \in J} p_\lambda$, $r_J(s) = s \wedge \bigvee_{\lambda \in J} q_\lambda$, and $d_J = \bigvee_{\lambda \in J} (p_\lambda \wedge q_\lambda)$. So K supports the family iff it contains a color with $p_\lambda = 1$ whenever there is one, a color with $q_\lambda = 1$ whenever there is one, and a color with $p_\lambda = q_\lambda = 1$ whenever there is one. If some color has $p_\lambda = q_\lambda = 1$, it alone supports the family; otherwise one color with $p_\lambda = 1$ and one with $q_\lambda = 1$, whenever such colors exist, together support it. The family $(p_\lambda, q_\lambda) = (1, 0), (0, 1)$ on two colors needs both of them, so $\text{sn}(\wedge) \geq 2$. $\square$

The *support number* of a computation $\mathcal{C}$ is the sum of the support numbers of its aggregation gates,

$$\text{sn}(\mathcal{C}) = \sum_{g \text{ an aggregation gate of } \mathcal{C}} \text{sn}(\odot_g),$$

so a CoR term or relational circuit with m compositions has support number $2m$.

3.4 Simultaneous preservation

As outlined in Section 1.4.5, choosing a support at every gate and taking the union preserves every aggregation of a computation at once.

Theorem 3.7 (Preservation). *Let $\mathcal{C}$ be a computation whose inputs are type-constant arrays on D . There is a set $K \subseteq \Lambda$ with $|K| \leq \text{sn}(\mathcal{C})$ such that for every nonempty J with $K \subseteq J \subseteq \Lambda$ and every gate g of $\mathcal{C}$,*

$$\text{value of } g \text{ on the inputs restricted to } D_J = (\text{value of } g \text{ on } D) \upharpoonright J.$$

In particular the outputs agree. Each aggregation gate is charged once, however often its value is used.

Proof. If $\text{sn}(\mathcal{C}) = \infty$, take $K = \Lambda$. Otherwise evaluate $\mathcal{C}$ on D . By Corollary 3.3, extended to arbitrary kernels as noted in Section 3.3, the value of every gate is type-constant. For each aggregation gate g with operands P, Q choose a support K_g of the family $(P(\tau_\lambda), Q(\tau_\lambda))_{\lambda \in \Lambda}$ with $|K_g| \leq \text{sn}(\odot_g)$, and let $K = \bigcup_g K_g$. Now fix a nonempty $J \supseteq K$ and proceed by induction along the evaluation order. For the inputs the claim holds by definition. A free gate commutes with restriction, so if its operands agree after restriction, so does its value. At an aggregation gate the operands agree by induction, $J \supseteq K_g$ supports its reference operands, and Lemma 3.4 gives the claim. The supports were chosen on the reference computation, independently of J ; this is why a single set K serves every $J \supseteq K$. $\square$

Two remarks are in order. First, the only closure property used is that of the values on D ; nothing is assumed about the restricted values. Second, for a CoR term or circuit with m compositions Lemma 3.6 gives $|K| \leq 2m$; complement gates need no separate treatment, since the theorem asserts an equality (Section 1.4.5).

The theorem reduces lower bounds to counting. Let φ be a binary query and let $\mathfrak{M}$ interpret its symbols by type-constant relations. A color λ is *detectable* for φ if

$$\varphi^{\mathfrak{M} \upharpoonright (\Lambda \setminus \{\lambda\})}(u, v) \neq \varphi^{\mathfrak{M}}(u, v) \quad \text{for some } u, v \in D_{\Lambda \setminus \{\lambda\}},$$

that is, if deleting the cell of color λ changes φ at a pair of surviving vertices.

Corollary 3.8 (Detectable colors). *Let $\Lambda_0 \subseteq \Lambda$ be a set of detectable colors for φ on $\mathfrak{M}$. Every computation $\mathcal{C}$ whose output equals φ on $\mathfrak{M}$ and on the $|\Lambda_0|$ structures $\mathfrak{M} \upharpoonright (\Lambda \setminus \{\lambda\})$, $\lambda \in \Lambda_0$, has $\text{sn}(\mathcal{C}) \geq |\Lambda_0|$. In particular, every CoR term or relational circuit that agrees with φ on these $|\Lambda_0| + 1$ finite structures has at least $\lceil |\Lambda_0|/2 \rceil$ compositions, since that count is an integer.*

Proof. If $\text{sn}(\mathcal{C}) < |\Lambda_0|$, the set K of Theorem 3.7 misses some $\lambda \in \Lambda_0$. For $J = \Lambda \setminus \{\lambda\}$, which is nonempty as $N \geq 2$, the output of $\mathcal{C}$ on $\mathfrak{M} \upharpoonright J$ is the restriction of its output on $\mathfrak{M}$, whereas φ changes at a pair of D_J . $\square$

For a non-Boolean S , “the output equals φ ” means that the output encodes the truth values of φ by two fixed distinct elements of S ; intermediate values may be arbitrary. In both interpretations of Section 4 every color is detectable, so $\Lambda_0 = \Lambda$ and the bound is $\lceil |\Lambda|/2 \rceil$; Remark B.5 in Appendix B is the one place where a single color has to be given up.

4 The two formulas

We now interpret the relation symbols of Ψ_k and Φ_k on the structure of Section 3 so that every color is detectable at a read-off pair, following the pattern of Section 1.4.2: the anchor supplies a

label, the hub supplies the *color* of the witness's cell, and the formula must recognize a single color; here labels and colors are subsets of $[k]$. Ψ_k tests an inclusion between the label and the color, so the labels are chosen to be of equal size, which turns inclusion into equality; Φ_k tests agreement directly, which allows every subset of $[k]$ to serve as a label.

4.1 Disjunctive clauses

We take as colors the $\lfloor k/2 \rfloor$ -element subsets of $[k]$, that is, $\Lambda = \binom{[k]}{\lfloor k/2 \rfloor}$, with one anchor x_A for each color A , and interpret each R_i as the type-constant relation with

$$R_i(\tau_{x_A,*}) = 1 \iff i \in A, \quad R_i(\tau_B) = 1 \iff i \notin B \quad (B \in \Lambda), \quad (12)$$

and with R_i false on every other type.

At the pair (x_A, h) , a cell witness $z \in C_B$ satisfies all k clauses iff $B \subseteq A$, that is, iff $B = A$, since A and B have the same size; the hub and the anchors are not witnesses, since R_i is false on (h, h) and on (x_A, x_B) while neither A nor B is all of $[k]$ (see Section 1.4.6 for the details). Hence, for every nonempty $J \subseteq \Lambda$ and every $A \in \Lambda$,

$$\Psi_k^{\mathfrak{M} \upharpoonright J}(x_A, h) \text{ holds} \iff A \in J. \quad (13)$$

Proof of Theorem 1.2. By (13), deleting the cell of color A changes Ψ_k at the surviving pair (x_A, h) , so every color is detectable and $\Lambda_0 = \Lambda$. A term or circuit that agrees with Ψ_k on all finite structures agrees with it in particular on $\mathfrak{M}$ and on the N structures $\mathfrak{M} \upharpoonright (\Lambda \setminus \{A\})$, so Corollary 3.8 gives $m \geq \lceil N/2 \rceil$ for its number m of compositions, where $N = \binom{k}{\lfloor k/2 \rfloor}$. Equation (5) gives $|t| \geq 2m + 1 \geq N + 1$. $\square$

Why the middle layer. The central binomial coefficient in Theorem 1.2 is not an artifact of the interpretation (12). Each clause of Ψ_k is a disjunction, so at a witness the k clauses amount to a disjointness test between two subsets of $[k]$, and detecting a color turns into a cross-intersecting condition on pairs of such subsets; Bollobás's set-pair inequality then shows that no type-constant interpretation of Ψ_k , over any set of colors and with any number of anchors, makes more than $\binom{k}{\lfloor k/2 \rfloor}$ colors detectable (Proposition B.3 in Appendix B). The interpretation (12) is thus optimal, and the loss of a factor $\Theta(\sqrt{k})$ against (3) is inherent to Ψ_k together with this construction; whether Ψ_k itself needs 2^k compositions we do not determine.

4.2 Profile agreement

For a pair (u, v) and an index $i \in [k]$ write

$$c_i(u, v) = (P_i(u, v), Q_i(u, v)) \in \mathbb{B}^2$$

for the *code* of i at (u, v) . The i -th clause of (4) at a witness z is the *agreement test*

$$\eta((p, q), (p', q')) = (p \wedge p') \vee (q \wedge q') \quad (14)$$

applied to $c_i(x, z)$ and $c_i(z, y)$. Writing

$$e_1 = (1, 0), \quad e_0 = (0, 1), \quad e_\perp = (0, 0),$$

for e, e' among these three codes, $\eta(e, e') = 1$ holds exactly when $e = e' \in \{e_0, e_1\}$; the *invalid* code $e_\perp$ agrees with no code, not even with itself.

Say that a type τ *carries the label* $A \subseteq [k]$ if $c_i(\tau) = e_1$ for every $i \in A$ and $c_i(\tau) = e_0$ for every $i \notin A$, and that it *carries no label* if $c_i(\tau) = e_\perp$ for some i . The interpretation below gives every type one of the three codes at every i , so these two cases are exhaustive. A witness z then satisfies all k clauses at (u, v) precisely when the types of its two edges carry labels and the two labels are equal.

Take $\Lambda = 2^{[k]}$, the set of *all* subsets of $[k]$, with one anchor x_A for every $A \in \Lambda$, and interpret the $2k$ symbols by the type-constant arrays with

$$\tau_{x_A, *} \text{ carries } A, \quad \tau_B \text{ carries } B \quad (B \in \Lambda), \quad (15)$$

and every other type carrying no label, that is $\tau_=$, each $\tau_{*, x}$ and each $\tau_{x, x'}$. In terms of the symbols: on $\tau_{x_A, *}$, P_i holds iff $i \in A$ and Q_i iff $i \notin A$; on τ_B , P_i holds iff $i \in B$ and Q_i iff $i \notin B$; and $P_i = Q_i = 0$ everywhere else.

Consider the pair (x_A, h) and a witness z .

- $z \in C_B$: the two edges carry the labels A and B , so z is a witness exactly when $B = A$.
- $z = h$: the second edge is the loop (h, h) , of type $\tau_=$, which carries no label, so no clause holds.
- $z = x'$, an anchor, possibly x_A itself: the first edge has type $\tau_{x_A, x'}$, which carries no label, so again no clause holds.

Hence, for every nonempty $J \subseteq \Lambda$ and every $A \in \Lambda$,

$$\Phi_k^{\mathfrak{M} \upharpoonright J}(x_A, h) \text{ holds} \iff A \in J. \quad (16)$$

Proof of Theorem 1.3. By (16) all $N = 2^k$ colors are detectable, so Corollary 3.8 gives $2m \geq 2^k$ for the number m of compositions, that is $m \geq 2^{k-1}$, and (5) gives $|t| \geq 2m + 1 \geq 2^k + 1$. The matching term is (17) below. $\square$

The hub and the anchors are excluded as witnesses not because their labels go unused but because they carry no label at all; this third code is exactly what the independence of P_i and Q_i provides, since the two symbols are complementary on the types that carry a label and both false elsewhere. The interpretation (15) is Boolean only in its values, and Section 6.2 reuses it verbatim over an arbitrary semiring.

The matching upper bound is built like (3), from one composition for each subset of $[k]$:

$$\text{CoR}_{\Phi_k} := \bigcup_{A \subseteq [k]} (H_A; H_A), \quad H_A = \left(\bigcap_{i \in A} P_i \right) \cap \left(\bigcap_{i \notin A} Q_i \right). \quad (17)$$

It has 2^k compositions: a witness satisfies clause i either through P_i or through Q_i , and collecting the indices of the first kind into a set A says precisely that both of its edges lie in H_A . The subset A plays the same role as I in (3), with one difference: there I distributes the symbols between the two edges of the witness, whereas here A selects a single relation that both edges must satisfy. Hence $\Phi_k \equiv \text{CoR}_{\Phi_k}$ on all structures, and (17) again has 2^k compositions and $O(k2^k)$ nodes.

Optimality and the factor of two. Appendix B shows that no type-constant interpretation of Φ_k makes more than 2^k colors detectable (Proposition B.4), so on this structure Corollary 3.8

cannot yield more than the 2^{k-1} compositions of Theorem 1.3. The remaining factor of two against (17) is therefore not a loss in the choice of labels but the support number $\text{sn}(\wedge) = 2$ of Lemma 3.6: a single composition can monitor two colors at once, and the family in the proof of that lemma shows that this cannot be improved per gate. Closing the factor would require a different invariant, not a different interpretation.

The interpretations (12) and (15) also make sense for a computation over an arbitrary S , with inputs and outputs encoded by two fixed distinct elements of S and arbitrary intermediate values; Corollary 3.8 then gives

$$\text{sn}(\mathcal{C}) \geq \binom{k}{\lfloor k/2 \rfloor} \text{ for } \Psi_k, \quad \text{sn}(\mathcal{C}) \geq 2^k \text{ for } \Phi_k.$$

The only role of conjunction in Theorems 1.2 and 1.3 is its support number two.

One feature of both families should be recorded, since it is what makes $|\varphi| = \Theta(k)$.

Remark 4.1 (The vocabulary grows with k). Ψ_k uses k relation symbols and Φ_k uses $2k$; the size $|\varphi|$ counts the nodes of the syntax tree and charges an occurrence of a symbol one node, so $|\varphi| = \Theta(k)$ in both cases and Corollary 1.4 holds as stated. Over a vocabulary *fixed* in advance the question is a different one, and we leave it open: we do not know whether an exponential succinctness gap holds for a family of FO^3 formulas over, say, a single binary relation symbol.

Part II: Additional results

5 Size-specific computations and bounded error

Restriction compares structures of different sizes. In this section we keep the domain fixed and allow the target to be chosen separately for each domain size. No new finite structure needs to be built, because of the way in which a deletion acted in the first place: by Section 3.3, a deletion affected a composition only through the three responses, that is, only by removing a color from the palette that the hub can see. Recoloring the hub's edges into C_λ with some other color has exactly the same effect and removes no vertex. The same three responses therefore control such a recoloring of $\mathfrak{M}$, on a domain that never changes.

5.1 Transport of type values on a fixed domain

For a map $f : \Lambda \rightarrow \Lambda$, we change only the colors of the edges at the hub:

$$\chi_f(h, u) = \chi_f(u, h) = f(\lambda) \quad (u \in C_\lambda), \quad \chi_f(u, u') = \chi(u, u') \quad (u \neq u' \in C_\Lambda),$$

and put $J = f(\Lambda)$. Write τ_λ^f for the new color types and tp_f for the new typing map; the anchor types and τ_+ are unchanged. Every color still occurs on edges between cell vertices, so these types are nonempty and partition D^2 . What is lost is homogeneity: seen from the hub, only the colors in J now occur, whereas every color still occurs elsewhere. Hence Lemma 3.2 fails for χ_f , and the type-constant arrays with respect to the recolored types need not be closed under aggregation. The lemma below replaces this closure property.

For a type-constant P , let f_*P be the array that takes the value $P(\tau_\lambda)$ on τ_λ^f and agrees with P on all other types. This map commutes with uniform pointwise functions, transpose, and the permitted constants. In particular, it does not change the values at the read-off pairs:

$$(f_*P)(x, h) = P(x, h) \quad (x \in X). \quad (18)$$

The map f_* transports values between types; it need not preserve individual entries elsewhere.

The three responses now have to be compatible with f_* rather than with restriction.

Lemma 5.1 (Three-response recoloring). *For type-constant P, Q , if the responses (10) agree on $J = f(\Lambda)$ and on Λ , then*

$$f_*(P ;_\odot Q) = (f_*P) ;_\odot (f_*Q).$$

Proof. Both sides are arrays on the same domain D ; we compare them at a pair (u, v) . We call τ the *name* of the recolored type of (u, v) : it is τ_λ when $\text{tp}_f(u, v) = \tau_\lambda^f$, and it is the type itself for the four kinds of types that χ_f leaves unchanged, namely $\tau_=$, the $\tau_{x,*}$, the $\tau_{*,x}$ and the $\tau_{x,x'}$. By the definition of f_* the left-hand side at (u, v) equals $(P ;_\odot Q)(\tau)$, which by (8) is the join of $P(\tau_1) \odot Q(\tau_2)$ over the reference profiles $(\tau_1, \tau_2) \in W_\tau$. The right-hand side at (u, v) is the join of the same expression over the names of the recolored profiles,

$$W_{(u,v)}^f = \{\text{names of } (\text{tp}_f(u, z), \text{tp}_f(z, v)) : z \in D\},$$

because $(f_*P)(u, z) = P(\tau_1)$ when τ_1 is the name of $\text{tp}_f(u, z)$, and likewise for Q . So it suffices to show that the two joins agree. Since χ_f differs from χ only on the edges at the hub, and there replaces the palette Λ by J , we sort the witnesses into three groups.

Anchor witnesses and endpoint witnesses. An anchor witness z gives two types among the anchor types, which χ_f does not touch, so it contributes its reference term. A witness $z \in \{u, v\} \cap U$ gives $(\tau_=, \tau)$ or $(\tau, \tau_=)$ if $u \neq v$, and $(\tau_=, \tau_=)$ if $u = v$; these are the reference profiles that W_τ attaches to the endpoints of a pair of type τ , because τ is by construction the name of the recolored type of (u, v) .

The hub as a witness, when neither endpoint is the hub. The hub then contributes exactly one profile name, and that name already occurs among the cell witnesses at the same pair, so idempotence of the join absorbs it. In detail, writing C_μ and C_ν for the cells of the left and the right endpoint when these are cell vertices: at two distinct cell endpoints the hub gives $(\tau_{f(\mu)}, \tau_{f(\nu)})$, and Lemma 3.1(a) realizes every ordered pair of colors there; at equal cell endpoints it gives a paired name $(\tau_{f(\mu)}, \tau_{f(\mu)})$, and Lemma 3.1(b) realizes every paired color; at an anchor–cell pair it gives $(\tau_{x,*}, \tau_{f(\nu)})$ or $(\tau_{f(\mu)}, \tau_{*,x'})$, and Lemma 3.1(b) realizes every color on the cell side; and at a pair of two anchors it gives $(\tau_{x,*}, \tau_{*,x'})$, which every core witness gives.

Cell witnesses. These are the entries of the table (11), read with $I = J$ and with the cell endpoints ranging over all of C_Λ . Indeed a witness $z \in C_\mu$ is now seen *from the hub* through the name $\tau_{f(\mu)}$, so as μ runs over Λ that name runs over J ; at any other endpoint every color is still available, by Lemma 3.1. The entries that depend on I are $r_I(p_x)$, $\ell_I(q_{x'})$, d_I , $\bigvee_\mu \ell_I(q_\mu)$ and $\bigvee_\mu r_I(p_\mu)$, and the hypothesis turns each of them into its value at $I = \Lambda$, which is the reference entry. The remaining entries do not depend on I .

The three groups exhaust D , and together they reproduce the join over W_τ at every pair (u, v) . Beyond the idempotence of the join, no law of $\odot$ is used. $\square$

As in Section 3.4, choosing a support at every gate handles an entire computation at once.

Theorem 5.2 (Simultaneous transport). *For a computation $\mathcal{C}$ with type-constant inputs on D , there is $K \subseteq \Lambda$ with $|K| \leq \text{sn}(\mathcal{C})$ such that every map $f : \Lambda \rightarrow \Lambda$ with $f(\Lambda) \supseteq K$ transports every intermediate value by f_* : the value of each gate on the inputs f_*P is f_* of its value on the original inputs P . One may use the same support as in Theorem 3.7.*

Proof. Use the support selected at each aggregation gate on the reference computation. Free operations commute with f_* and Lemma 5.1 handles aggregation, so induction in evaluation order gives the claim. If the support number is infinite, use $K = \Lambda$ and the same argument, since $f(\Lambda) = \Lambda$ preserves every response. Each shared gate is charged once. $\square$

All comparisons now take place on one and the same domain D . The wiring, kernels, scalar constants, and pointwise functions may therefore have been chosen for that domain size, provided that they are fixed before the input is given and are uniform across entries. Besides the inputs, the arrays available as primitives remain those listed in Section 3.3, namely the constant arrays and the equality array. What is ruled out is any further primitive array whose (u, v) entry varies with u and v , such as an array recording the cell of each vertex, since such an array would give the computation exactly the information that a type hides.

5.2 Detectable labels at a fixed size

For $f : \Lambda \rightarrow \Lambda$, let $\mathfrak{M}_f$ be the structure on D whose relations are $f_*R^{\mathfrak{M}}$, that is, the reference structure with its hub edges recolored. Suppose that a Boolean-output query φ has a type-constant interpretation on $\mathfrak{M}$ together with a set $\Lambda_0 \subseteq \Lambda$ of labels, with corresponding anchors x_λ , that are *detectable under recoloring* in the following sense: for every $\lambda \in \Lambda_0$,

$$\varphi^{\mathfrak{M}_f}(x_\lambda, h) \text{ holds} \iff \lambda \in f(\Lambda). \quad (19)$$

For a non-Boolean S , the two truth values are represented by two fixed distinct output states.

For each $\lambda \in \Lambda_0$, fix a color $\lambda' \neq \lambda$ and let $f_\lambda : \Lambda \rightarrow \Lambda$ be the map that fixes all other colors and sends λ to λ' ; then $f_\lambda(\Lambda) = \Lambda \setminus \{\lambda\}$, and $\mathfrak{M}_{f_\lambda}$ is $\mathfrak{M}$ with the hub edges into C_λ recolored λ' . All of these structures have the same domain D , and they are the fixed-size counterparts of the deletions used in Corollary 3.8.

Proposition 5.3 (A fixed-size test). *Under these assumptions, any aggregation computation $\mathcal{C}$ agreeing with φ on $\mathfrak{M}$ and on the $|\Lambda_0|$ structures $\mathfrak{M}_{f_\lambda}$, $\lambda \in \Lambda_0$, has $\text{sn}(\mathcal{C}) \geq |\Lambda_0|$.*

Proof. If $\text{sn}(\mathcal{C}) < |\Lambda_0|$, a support from Theorem 5.2 omits some $\lambda \in \Lambda_0$. The structures $\mathfrak{M}$ and $\mathfrak{M}_{f_\lambda}$ then give the same output at (x_λ, h) by (18), but opposite correct answers by (19). These $|\Lambda_0| + 1$ structures are fixed independently of the candidate. $\square$

For both logical families, every color is detectable under recoloring, so $\Lambda_0 = \Lambda$, and the number of colors is

$$N(\Psi_k) = \binom{k}{\lfloor k/2 \rfloor} \quad (k \geq 2), \quad N(\Phi_k) = 2^k \quad (k \geq 1). \quad (20)$$

Each of the structures involved has

$$n_0(N) = N + 1 + N \lceil 12N^2 \ln(2N) \rceil = O(N^3 \log N) \quad (21)$$

elements: N anchors, the hub, and N cells of L vertices each. Both values of N in (20) are exponential in k , and so is the threshold: $n_0 = 2^{O(k)}$ for either family. Consequently, nothing below applies to a target that only has to be correct on structures whose size is polynomial in k , and the construction cannot be adjusted to supply such structures: property (a) of Lemma 3.1 already forces $L \geq N^2 + 1$ (Remark A.2), so structures of the kind used here have at least N^3 vertices however the coloring is chosen.

Both families satisfy (19) once the domain has been padded with inert anchors, as the next theorem verifies. That theorem and Corollary 5.5 below are the precise form of Theorem 1.5.

Theorem 5.4 (Size-specific lower bounds). *Let φ be Ψ_k or Φ_k , let $N(\varphi)$ be as in (20), and let $n \geq n_0(N(\varphi))$ with n_0 as in (21). All the composition-count and term-size lower bounds of Theorems 1.2 and 1.3 hold for targets that are required to be correct only on n -element inputs, even when the target depends arbitrarily on k and n . More generally, let $\mathcal{C}$ be an aggregation computation of this Boolean query in the sense of Section 3.3, with arbitrary intermediate scalar values, whose wiring, kernels, scalar constants and pointwise maps may all have been chosen for k and n , and whose primitive arrays are the inputs of φ together with the constant arrays and the equality array, and nothing else. Then $\text{sn}(\mathcal{C}) \geq N(\varphi)$.*

The last restriction is essential; see the discussion after Theorem 5.2.

Proof. Use the corresponding interpretation of Section 4 and add anchors until the domain has size n ; this is possible for every n in the stated range, since the initial domain has $n_0(N)$ elements. The extra anchors are made inert as follows. For Ψ_k , set every entry incident to an extra anchor to zero, so that such a witness has both of its edges false at a read-off pair. For Φ_k , give every entry incident to an extra anchor the code $e_\perp$, so that no type at such an anchor carries a label and such a witness fails every clause; the core inputs of (15) are kept unchanged. In both cases the original anchors and the hub remain excluded as witnesses, for the reasons given in Section 4, and all the inputs remain type-constant.

It remains to check (19). For both families a color is a subset of $[k]$. After recoloring by f , a witness in the original cell C_B is seen from the hub through $f(B)$ rather than B , so for Ψ_k it satisfies the clauses at x_A exactly when $f(B) \subseteq A$, hence exactly when $f(B) = A$, and for Φ_k it succeeds at x_A exactly when $f(B) = A$. Either way the pair (x_λ, h) is satisfied precisely when $\lambda \in f(\Lambda)$, which is (19) for every $\lambda \in \Lambda$. Proposition 5.3 now gives the support-number bound. For CoR we have $\text{sn}(\mathcal{C}) = 2m$, so $m \geq \lceil N(\varphi)/2 \rceil$ since m is an integer, and Equation (5) gives the term-size bounds. $\square$

The explicit upper bounds are valid at every size, so the composition count for Φ_k remains $\Theta(2^k)$ even when the target may depend on the size. This is a result about relational circuits, not a lower bound for unrestricted Boolean circuits with access to individual input entries.

5.3 A bounded-error consequence

For fixed k and n , a randomized computation is an input-independent probability distribution over computations in the sense of Section 3.3, each of which has its wiring, kernels, constants and

pointwise maps fixed. The distribution may depend on k and n . The error is measured at each input and each output pair. Random auxiliary relations, random vertex labels, and input-dependent choices of a circuit are not part of this model: the first two are in general not type-constant, so Theorem 5.2 does not apply to a computation that has one of them among its primitives (see the discussion after that theorem). The distribution may thus range over the computation, but not over the arrays from which it starts.

Corollary 5.5 (Bounded error at a fixed size). *Let φ be either family and $n \geq n_0(N(\varphi))$. A randomized relational circuit computing φ with error at most $\varepsilon < 1/2$ on every input and output pair satisfies*

$$\mathbb{E}[g_i(C)] \geq \frac{1 - 2\varepsilon}{2} N(\varphi).$$

The same holds for expected composition occurrences in terms. For general aggregation computations the bound is $\mathbb{E}[\text{sn}(C)] \geq (1 - 2\varepsilon)N(\varphi)$.

Proof. Choose a label λ uniformly from Λ and, with equal probability, test (x_λ, h) on $\mathfrak{M}$ or on $\mathfrak{M}_{f_\lambda}$; the correct answers are opposite. For a deterministic computation with support K , every $\lambda \notin K$ gives the same output on the two tests, so at most one of them can be correct, and the success probability is at most

$$\frac{1}{2} + \frac{|K|}{2N(\varphi)} \leq \frac{1}{2} + \frac{\text{sn}(C)}{2N(\varphi)}.$$

Averaging over the input-independent distribution, a pointwise error of at most ε gives an average success probability of at least $1 - \varepsilon$, and comparing this with the bound above yields the claim. If the expected support number is infinite, the claim is immediate. For relational circuits, use $\text{sn}(C) = 2g_i(C)$. $\square$

In particular, error $1/3$ still requires $N(\varphi)/6$ compositions in expectation, that is $2^k/6$ for Φ_k .

6 Non-Boolean aggregation and matrix query languages

The preservation theorem of Section 3 was proved for an arbitrary kernel. We now compute the support number of semiring multiplication and apply the theorem to arity elimination.

6.1 The support number of semiring multiplication

An idempotent semiring is a tuple $\mathbb{S} = (S, \vee, \otimes, 0_{\mathbb{S}}, 1_{\mathbb{S}})$: addition is a commutative idempotent monoid, multiplication is a monoid, $0_{\mathbb{S}}$ is an annihilator, and multiplication distributes over finite joins on both sides. Assume $0_{\mathbb{S}} \neq 1_{\mathbb{S}}$; multiplication need not be commutative unless stated. The *join breadth* $\beta(\mathbb{S})$ is the least β such that every finite join is the join of at most β of its summands, or ∞ . An idempotent semiring whose addition orders S totally — $a \leq b$ when $a \vee b = b$ — has breadth one, since a finite join is then its largest summand.

Semiring matrix multiplication is precisely the aggregation (9) with kernel $\otimes$; we write $P \star Q$ for $P ;_{\otimes} Q$. The equality array is the usual identity matrix I_D . The model includes arbitrary uniform pointwise functions, constant-valued matrices, transpose, and shared intermediates.

For this kernel the support number is governed by the join breadth.

Proposition 6.1 (Semiring responses). *For semiring multiplication, the join of Definition 3.5 being the semiring addition $\vee$,*

$$\beta(\mathbb{S}) \leq \text{sn}(\otimes) \leq 3\beta(\mathbb{S}).$$

In particular, if $\beta < \infty$, then every computation with m products has a preserving support of at most $3\beta m$ colors, both under restriction and under recoloring.

Proof. Distributivity gives

$$\ell_J(s) = \left(\bigvee_{\lambda \in J} p_\lambda \right) \otimes s, \quad r_J(s) = s \otimes \left(\bigvee_{\lambda \in J} q_\lambda \right),$$

and evaluation at $1_{\mathbb{S}}$ recovers the two marginal joins. The triple of responses is therefore equivalent to the three scalar values

$$\bigvee_{\lambda \in J} p_\lambda, \quad \bigvee_{\lambda \in J} q_\lambda, \quad \bigvee_{\lambda \in J} p_\lambda \otimes q_\lambda,$$

and it suffices to choose at most β colors for each of them. For the lower bound on $\text{sn}(\otimes)$, set every $q_\lambda = 1_{\mathbb{S}}$; then the paired response becomes $d_J = \bigvee_{\lambda \in J} p_\lambda$, so a support K must already satisfy $\bigvee_{\lambda \in K} p_\lambda = \bigvee_{\lambda \in \Lambda} p_\lambda$, and a finite family (p_λ) whose join is not the join of fewer than β of its summands therefore needs β colors. The preservation statements are those of Theorems 3.7 and 5.2, and require no new closure or witness calculation. $\square$

The lower inequality also shows that, for these unital kernels, a finite support number is equivalent to finite join breadth. Neither inequality is tight in general: Boolean conjunction has $\beta = 1$ but $\text{sn}(\wedge) = 2$, by Lemma 3.6, whereas over max-plus, min-plus and max-min the upper bound is attained, $\text{sn}(\otimes) = 3$ (Appendix C), so over these semirings the factor 3β in Theorems 6.2 and 6.4 is the support number itself. Under restriction the scalar maps are always held fixed; size-specific maps are allowed in the recoloring comparison, which takes place on a single domain.

6.2 Profile agreement over a semiring

A general semiring has no Boolean equivalence, but the agreement test (14) of Section 4.2 is already algebraic. We read the three codes in $\mathbb{S}$,

$$e_1 = (1_{\mathbb{S}}, 0_{\mathbb{S}}), \quad e_0 = (0_{\mathbb{S}}, 1_{\mathbb{S}}), \quad e_{\perp} = (0_{\mathbb{S}}, 0_{\mathbb{S}}),$$

and let $\eta((p, q), (p', q')) = (p \otimes p') \vee (q \otimes q')$. Then $\eta(e, e') = 1_{\mathbb{S}}$ if $e = e' \in \{e_0, e_1\}$ and $0_{\mathbb{S}}$ if $e \neq e'$, while the invalid code $e_{\perp}$ matches nothing. The codes are pairs of input values, not an extension of the scalar state space.

For a commutative idempotent semiring, we use the $2k$ independent inputs $P_1, Q_1, \dots, P_k, Q_k$ and define

$$F_k(x, y) = \bigvee_z \bigotimes_{i=1}^k (P_i(x, z) \otimes P_i(z, y) \vee Q_i(x, z) \otimes Q_i(z, y)), \quad (22)$$

whose clause is η applied to the codes of the two edges. Over the Boolean semiring, (22) is literally Φ_k . Over a general $\mathbb{S}$ the pairs (P_i, Q_i) remain independent, and (22) defines an algebraic query on arbitrary semiring-valued inputs rather than a Boolean equivalence.

An annotated relation assigns a scalar to each tuple. In positive annotated relational algebra, union adds annotations, natural join multiplies them, projection sums over removed coordinates, and renaming changes attribute names. Following Brijder et al. [7], ARA(3) denotes this algebra restricted to expressions all of whose intermediate relations have at most three attributes. The query F_k has an ARA(3) expression of size $O(k)$: for each i , join copies of P_i renamed to (x, z) and (z, y) , do the same for Q_i , and take their union; then join the k ternary results and project away z . Every intermediate has at most three attributes.

The interpretation of Section 4.2 and Proposition 6.1 now give the lower bound.

Theorem 6.2 (Weighted-query lower bound). *Let $\mathbb{S}$ be a nontrivial commutative idempotent semiring of finite join breadth β , and let $\mathcal{C}$ be a square-matrix computation of F_k on all finite inputs in which every aggregation gate carries the kernel $\otimes$. Writing m for the number of these gates, the products of $\mathcal{C}$, we have*

$$m \geq \frac{2^k}{3\beta} \quad (k \geq 1).$$

Conversely, 2^k products suffice, with $O(k2^k)$ tree nodes or $O(2^k)$ nodes with sharing. The same lower bound holds separately at every fixed size $n \geq n_0(2^k)$, even with size-specific scalar maps and wiring.

Proof. Use the interpretation (15) of Section 4.2 with the codes read in $\mathbb{S}$: all $N = 2^k$ subsets of $[k]$ serve as colors, $\Lambda = 2^{[k]}$, with one anchor x_A per color; the type $\tau_{x_A, *}$ carries the label A , the type τ_B carries B , and every other type carries no label. All inputs are type-constant. At (x_A, h) a cell witness succeeds exactly when its color is A , while an anchor witness and the hub have an unlabelled type on one of their edges, so, for every nonempty $J \subseteq \Lambda$, every $f : \Lambda \rightarrow \Lambda$ and every $A \in \Lambda$,

$$F_k^{\mathfrak{M} \upharpoonright J}(x_A, h) = 1_{\mathbb{S}} \iff A \in J, \quad F_k^{\mathfrak{M} f}(x_A, h) = 1_{\mathbb{S}} \iff A \in f(\Lambda), \quad (23)$$

and the value is $0_{\mathbb{S}}$ otherwise. All N labels are detectable.

Every aggregation gate of $\mathcal{C}$ costs at most 3β support colors by Proposition 6.1, and there are m of them, so $\text{sn}(\mathcal{C}) \leq 3\beta m$ and the set K of Theorem 3.7 has $|K| \leq 3\beta m$. If $3\beta m < N$ then K misses some color A , and deleting the cell C_A leaves the output at (x_A, h) unchanged, whereas F_k changes there by (23). For the fixed-size claim, pad with anchors carrying no label on any incident type and apply Proposition 5.3. All intermediate values may be arbitrary scalars, not just encoded bits.

For the upper bound let $H_A = \otimes_{i \in A} P_i \otimes \otimes_{i \notin A} Q_i$ entrywise. Distributivity and commutativity give

$$F_k = \bigvee_{A \subseteq [k]} H_A \star H_A, \quad (24)$$

which is (17) with $\otimes$ for $\cap$ and $\star$ for the composition. There are 2^k products and $O(k2^k)$ tree nodes; sharing prefix products gives $O(2^k)$ nodes. $\square$

The lower-bound interpretation uses only the two valid codes and the invalid one, so it also works with noncommutative multiplication if the source product is given its displayed order; commutativity is used only for the upper bound on general scalar inputs.

The hypothesis that every aggregation gate carries the kernel $\otimes$ is not merely cosmetic. A computation in the sense of Section 3.3 may use a different kernel at each gate, and the support

number of a computation is the sum over *all* of its aggregation gates (Definition 3.5); a gate carrying some other kernel therefore contributes its own support number to this sum, and a kernel of infinite support number defeats the count altogether. Appendix C exhibits such a kernel, together with a single aggregation that uses it to compute Φ_k outright. What the theorem counts is the number of products in a computation whose only aggregations are products, and this is exactly the form needed for Theorem 6.4: by Lemma 6.3, a MATLANG computation translates into such a computation, its products contracting the dimension 1 having become free entrywise gates.

The following examples all have breadth one.

Semiring	Scalars	$\vee$	$\otimes$	$(0_S, 1_S)$
Max-plus	$\mathbb{R} \cup \{-\infty\}$	max	+	$(-\infty, 0)$
Min-plus	$\mathbb{R} \cup \{+\infty\}$	min	+	$(+\infty, 0)$
Max-min	$[0, 1]$	max	min	$(0, 1)$

For instance the min-plus query is

$$\min_z \sum_{i=1}^k \min\{P_i(x, z) + P_i(z, y), Q_i(x, z) + Q_i(z, y)\},$$

and the minimum number of min-plus products needed to compute it lies between $2^k/3$ and 2^k . These are not ordinary arithmetic matrix products; the latter, combined with arbitrary decoding functions, are not covered by this theorem.

6.3 Typed MATLANG and arity elimination

Semiring MATLANG [7], starting from square inputs of type $n \times n$, has intermediate types $n \times n$, $n \times 1$, $1 \times n$, and 1×1 . Besides entrywise operations and transpose it has ones-vectors, column diagonalization, and conformable matrix multiplication. We allow arbitrary uniform entrywise functions.

Typed MATLANG is not literally the model of Section 3.3, but it embeds into that model at no cost.

Lemma 6.3 (Broadcasting typed intermediates). *A typed MATLANG computation can be represented in the square-matrix model without increasing its number of products, preserving sharing. Products contracting dimension n become one square product, and those contracting dimension 1 become entrywise operations.*

Proof. Encode matrices, columns, rows, and scalars, respectively, as

$$\widehat{A}(u, v) = A(u, v), \quad \widehat{p}(u, v) = p(u), \quad \widehat{w}(u, v) = w(v), \quad \widehat{s}(u, v) = s,$$

where the letters A, p, w, s are local to this proof. The scalar encoding stores the computed scalar at every entry; it is not an input-independent free constant. Transpose and pointwise operations respect the encodings. A ones-vector becomes the constant matrix 1_S . Diagonalization of a column multiplies its encoding entrywise by I_D ; diagonalization of a scalar leaves its encoding unchanged. A product contracting n sums over the common coordinate and is one square semiring product. A

product contracting 1 has one summand at each entry and is entrywise $\otimes$. These identities cover all conformable type combinations and keep the factor order. Translating each node once preserves sharing. $\square$

ARA(3) allows annotated relations of intermediate arity at most three, whereas MATLANG uses matrices, vectors, and scalars. The following is the precise form of Theorem 1.6.

Theorem 6.4 (Exponential cost of arity elimination). *Fix a nontrivial commutative idempotent semiring of finite join breadth β . For every $k \geq 1$ there is a binary-output positive ARA(3) expression of size $O(k)$ such that every equivalent MATLANG computation, even with arbitrary uniform entrywise functions and sharing, uses at least $2^k/(3\beta)$ matrix products that contract the dimension n ; an equivalent MATLANG expression with 2^k products exists. Equivalence is on all finite inputs over the fixed semiring.*

Proof. Use the ARA(3) expression for F_k and apply Lemma 6.3 to any equivalent MATLANG computation. The translated computation has $\otimes$ at every aggregation gate, as Theorem 6.2 requires, because every product contracting the dimension 1 has become a free entrywise gate; that theorem then gives the lower bound, which counts only the products contracting the dimension n , as stated. Expansion (24) is already a MATLANG expression and gives the upper bound. The same proof applies at every fixed $n \geq n_0(2^k)$, even when the scalar constants and entrywise maps are chosen for that size, since they remain uniform across entries. $\square$

Brijder et al. [7] establish the corresponding expressive equivalence, and the remark following Corollary 8 in the earlier version [6, p. 9] asks whether the exponential blow-up in arity elimination is necessary. The theorem shows that it is, for every fixed nontrivial commutative idempotent semiring of finite breadth, including the three examples above, and even for the enlarged target language.

6.4 Positive computations over further semirings

A separate transfer argument does not require idempotence. Here *positive* MATLANG uses only entrywise semiring addition and multiplication, transpose, ones-vectors, diagonalization, and matrix multiplication; scalar constants must lie in the subsemiring generated by $0_{\mathbb{S}}$ and $1_{\mathbb{S}}$, and arbitrary entrywise functions are not allowed.

On Boolean inputs such a computation is a Boolean one in disguise.

Corollary 6.5 (Positive transfer). *Let $\mathbb{S}$ be a nontrivial commutative semiring with $r \cdot 1_{\mathbb{S}} \neq 0_{\mathbb{S}}$ for every positive integer r . Interpret (22) using its sums and products. Every positive MATLANG computation of this query has at least 2^{k-1} products. This holds with sharing, on all finite inputs, and separately at every fixed size $n \geq n_0(2^k)$.*

Proof. Consider the subsemiring $S_0 = \{r \cdot 1_{\mathbb{S}} : r \geq 0\}$ and the map $S_0 \rightarrow \mathbb{B}$, $a \mapsto [a \neq 0_{\mathbb{S}}]$. It sends $0_{\mathbb{S}}$ to 0 and, by the assumption on $\mathbb{S}$, it sends $r \cdot 1_{\mathbb{S}}$ to 1 for every $r \geq 1$; without that assumption a positive multiple of $1_{\mathbb{S}}$ could equal $0_{\mathbb{S}}$ and the two clauses would conflict. The identities for integer sums and products then show that this map is a semiring homomorphism.

On Boolean inputs all intermediate values lie in S_0 , so applying the homomorphism entrywise Booleanizes the computation without changing its number of products. After Lemma 6.3 this is a

Boolean relational computation of the Boolean version of F_k , which is Φ_k itself. Theorem 1.3, or Theorem 5.4 at a fixed dimension, gives the bound. All transformations preserve sharing. $\square$

This includes $\mathbb{N}, \mathbb{Z}, \mathbb{R}, \mathbb{C}$ with the specified positive operations and constants, but not negative constants, subtraction, or arbitrary decoding functions. Expansion (24) still uses 2^k products: it needs distributivity and commutativity, not idempotence.

Wider scalars and other kernels. The invariant also measures what wider scalars and other kernels can buy. Appendix C shows that b -bit scalars with coordinatewise Boolean operations reduce the number of products needed for Φ_k by a factor of b and no more (Proposition C.1); that every kernel on $[0, 1]$ that is monotone in each argument has support number at most three, so the lower bounds persist with a factor $1/3$; and that a kernel of unbounded support number, bitwise AND on the nonnegative integers, computes Φ_k with a single aggregation. What a finite support number limits is the way in which encoded states can interact through a witness.

References

- [1] Micah Adler and Neil Immerman. An $n!$ lower bound on formula size. *ACM Transactions on Computational Logic*, 4(3):296–314, 2003. doi:10.1145/772062.772064.
- [2] Vince Bárány, Balder ten Cate, and Martin Otto. Queries with guarded negation. *Proceedings of the VLDB Endowment*, 5(11):1328–1339, 2012. doi:10.14778/2350229.2350250.
- [3] Béla Bollobás. On generalized graphs. *Acta Mathematica Academiae Scientiarum Hungaricae*, 16(3–4):447–452, 1965. doi:10.1007/BF01904851.
- [4] Geraldine Brady. *From Peirce to Skolem: A Neglected Chapter in the History of Logic*, volume 4 of *Studies in the History and Philosophy of Mathematics*. North-Holland, Amsterdam, 2000.
- [5] Robert Brijder, Floris Geerts, Jan Van den Bussche, and Timmy Weerwag. On the expressive power of query languages for matrices. *ACM Transactions on Database Systems*, 44(4):15:1–15:31, 2019. Conference version in ICDT 2018.
- [6] Robert Brijder, Marc Gyssens, and Jan Van den Bussche. On matrices and K -relations, 2019. URL <https://arxiv.org/abs/1904.03934v1>. arXiv:1904.03934v1.
- [7] Robert Brijder, Marc Gyssens, and Jan Van den Bussche. On matrices and K -relations. *Annals of Mathematics and Artificial Intelligence*, 90(2–3):181–210, 2022. doi:10.1007/s10472-021-09760-4.
- [8] Karl Bringmann, Nick Fischer, and Marvin Künnemann. A fine-grained analogue of Schaefer’s theorem in P: Dichotomy of $\exists^k\forall$ -quantified first-order graph properties. In *34th Computational Complexity Conference (CCC)*, volume 137 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:27. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019. doi:10.4230/LIPIcs.CCC.2019.31.
- [9] Stanley Burris and Javier Legris. The algebra of logic tradition. The Stanford Encyclopedia of Philosophy, edited by Edward N. Zalta, 2021. URL <https://plato.stanford.edu/entries/algebra-logic-tradition/>. Substantive revision of 12 February 2021.

- [10] K. A. Bush. Orthogonal arrays of index unity. *The Annals of Mathematical Statistics*, 23(3): 426–434, 1952. doi:10.1214/aoms/1177729387.
- [11] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970. doi:10.1145/362384.362685.
- [12] E. F. Codd. Relational completeness of data base sublanguages. In Randall J. Rustin, editor, *Data Base Systems*, volume 6 of *Courant Computer Science Symposia*, pages 65–98. Prentice-Hall, Englewood Cliffs, NJ, 1972.
- [13] Anuj Dawar, Martin Grohe, Stephan Kreutzer, and Nicole Schweikardt. Model theory makes formulas large. In *Automata, Languages and Programming, 34th International Colloquium (ICALP 2007)*, volume 4596 of *Lecture Notes in Computer Science*, pages 913–924. Springer, 2007.
- [14] Kousha Etessami, Moshe Y. Vardi, and Thomas Wilke. First-order logic with two variables and unary temporal logic. *Information and Computation*, 179(2):279–295, 2002. doi:10.1006/inco.2001.2953.
- [15] William Ewald and Wilfried Sieg, editors. *David Hilbert’s Lectures on the Foundations of Arithmetic and Logic 1917–1933*, volume 3 of *David Hilbert’s Lectures on the Foundations of Mathematics and Physics, 1891–1933*. Springer, Berlin and Heidelberg, 2013. doi:10.1007/978-3-540-69444-1.
- [16] George H. L. Fletcher, Marc Gyssens, Dirk Leinders, Dimitri Surinx, Jan Van den Bussche, Dirk Van Gucht, Stijn Vansummen, and Yuqing Wu. Relative expressive power of navigational querying on graphs. *Information Sciences*, 298:390–406, 2015. doi:10.1016/j.ins.2014.11.031.
- [17] Tim French, Wiebe van der Hoek, Petar Iliev, and Barteld Kooi. On the succinctness of some modal logics. *Artificial Intelligence*, 197:56–85, 2013. doi:10.1016/j.artint.2013.02.003.
- [18] Jiawei Gao and Russell Impagliazzo. The fine-grained complexity of strengthenings of first-order logic. Report TR19-009, Electronic Colloquium on Computational Complexity, 2019. URL <https://eccc.weizmann.ac.il/report/2019/009/>.
- [19] Jiawei Gao, Russell Impagliazzo, Antonina Kolokolova, and Ryan Williams. Completeness for first-order properties on sparse structures with algorithmic applications. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2162–2181. SIAM, 2017. doi:10.1137/1.9781611974782.141.
- [20] Floris Geerts, Thomas Muñoz, Cristian Riveros, Jan Van den Bussche, and Domagoj Vrgoč. Matrix query languages. *SIGMOD Record*, 50(3):6–19, 2021. doi:10.1145/3503780.3503782.
- [21] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS 2007)*, pages 31–40. ACM, 2007.
- [22] Martin Grohe and Nicole Schweikardt. The succinctness of first-order logic on linear orders. *Logical Methods in Computer Science*, 1(1:6):1–25, 2005. doi:10.2168/LMCS-1(1:6)2005. Conference version in LICS 2004.

- [23] A. S. Hedayat, Neil J. A. Sloane, and John Stufken. *Orthogonal Arrays: Theory and Applications*. Springer Series in Statistics. Springer, New York, 1999. doi:10.1007/978-1-4612-1478-6.
- [24] Lauri Hella and Miikka Vilander. Formula size games for modal logic and μ -calculus. *Journal of Logic and Computation*, 29(8):1311–1344, 2019. doi:10.1093/logcom/exz025.
- [25] Jelle Hellings, Catherine L. Pilachowski, Dirk Van Gucht, Marc Gyssens, and Yuqing Wu. From relation algebra to semi-join algebra: An approach to graph query optimization. *The Computer Journal*, 64(5):789–811, 2021. doi:10.1093/comjnl/bxaa031.
- [26] Jelle Hellings, Dirk Van Gucht, Marc Gyssens, and Yuqing Wu. The power of Tarski’s relation algebra on trees. *Journal of Logical and Algebraic Methods in Programming*, 126:100748, 2022. doi:10.1016/j.jlamp.2022.100748.
- [27] David Hilbert and Wilhelm Ackermann. *Grundzüge der theoretischen Logik*, volume 27 of *Die Grundlehren der mathematischen Wissenschaften*. Julius Springer, Berlin, 1928.
- [28] Mark Jerrum and Marc Snir. Some exact complexity results for straight-line computations over semirings. *Journal of the ACM*, 29(3):874–897, 1982.
- [29] Stasys Jukna. *Boolean Function Complexity: Advances and Frontiers*, volume 27 of *Algorithms and Combinatorics*. Springer, Berlin and Heidelberg, 2012. doi:10.1007/978-3-642-24508-4.
- [30] Gyula O. H. Katona. Solution of a problem of A. Ehrenfeucht and J. Mycielski. *Journal of Combinatorial Theory, Series A*, 17(2):265–266, 1974. doi:10.1016/0097-3165(74)90018-1.
- [31] Leslie R. Kerr. *The Effect of Algebraic Structure on the Computational Complexity of Matrix Multiplication*. PhD thesis, Cornell University, 1970.
- [32] Stephen Cole Kleene. *Introduction to Metamathematics*, volume 1 of *Bibliotheca Mathematica*. North-Holland and P. Noordhoff, Amsterdam and Groningen, 1952.
- [33] Rudolf Lidl and Harald Niederreiter. *Finite Fields*, volume 20 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, Cambridge, 2nd edition, 1997. doi:10.1017/CBO9780511525926.
- [34] Yuri V. Linnik. On the least prime in an arithmetic progression. I. The basic theorem. *Recueil Mathématique (Matematicheskii Sbornik) N.S.*, 15(57)(2):139–178, 1944.
- [35] Leopold Löwenheim. Über Möglichkeiten im Relativkalkül. *Mathematische Annalen*, 76(4):447–470, 1915. doi:10.1007/BF01458217. Page references are to the English translation, *On possibilities in the calculus of relatives*, in van Heijenoort (ed.), *From Frege to Gödel*, Harvard University Press, 1967, pp. 228–251.
- [36] Roger D. Maddux. The origin of relation algebras in the development and axiomatization of the calculus of relations. *Studia Logica*, 50(3–4):421–455, 1991. doi:10.1007/BF00370681.
- [37] Roger D. Maddux. *Relation Algebras*, volume 150 of *Studies in Logic and the Foundations of Mathematics*. Elsevier, Amsterdam, 2006.

- [38] Kurt Mehlhorn and Zvi Galil. Monotone switching circuits and Boolean matrix product. *Computing*, 16(1–2):99–111, 1976.
- [39] Gregory H. Moore. The emergence of first-order logic. In William Aspray and Philip Kitcher, editors, *History and Philosophy of Modern Mathematics*, volume 11 of *Minnesota Studies in the Philosophy of Science*, pages 95–135. University of Minnesota Press, Minneapolis, 1988.
- [40] Augustus De Morgan. On the syllogism, no. II. *Transactions of the Cambridge Philosophical Society*, 9:79–127, 1856. Read 25 February 1850.
- [41] Augustus De Morgan. On the syllogism, no. IV, and on the logic of relations. *Transactions of the Cambridge Philosophical Society*, 10:331–358, 1864. Read 23 April 1860. Reprinted in Heath (ed.), *On the Syllogism and Other Logical Writings*, Routledge and Kegan Paul, 1966.
- [42] Yoshiki Nakamura. Expressive power and succinctness of the positive calculus of binary relations. *Journal of Logical and Algebraic Methods in Programming*, 127:100760, 2022. doi:10.1016/j.jlamp.2022.100760.
- [43] Michael S. Paterson. Complexity of monotone networks for Boolean matrix product. *Theoretical Computer Science*, 1(1):13–20, 1975.
- [44] Charles S. Peirce. Description of a notation for the logic of relatives, resulting from an amplification of the conceptions of Boole’s calculus of logic. *Memoirs of the American Academy of Arts and Sciences*, 9:317–378, 1870.
- [45] Charles S. Peirce. On the algebra of logic. *American Journal of Mathematics*, 3(1):15–57, 1880. doi:10.2307/2369442.
- [46] Vaughan R. Pratt. The power of negative thinking in multiplying Boolean matrices. *SIAM Journal on Computing*, 4(3):326–330, 1975.
- [47] Vaughan R. Pratt. Origins of the calculus of binary relations. In *Proceedings of the 7th Annual IEEE Symposium on Logic in Computer Science*, pages 248–254, Santa Cruz, CA, 1992. IEEE Computer Society. doi:10.1109/LICS.1992.185537.
- [48] Walter J. Savitch. Relationships between nondeterministic and deterministic tape complexities. *Journal of Computer and System Sciences*, 4(2):177–192, 1970. doi:10.1016/S0022-0000(70)80006-X.
- [49] Uwe Schöning. *Logic for Computer Scientists*, volume 8 of *Progress in Computer Science and Applied Logic*. Birkhäuser, Boston, 1989. English translation of *Logik für Informatiker*, B.I.-Wissenschaftsverlag, 1987.
- [50] Ernst Schröder. *Vorlesungen über die Algebra der Logik (Exakte Logik), Dritter Band: Algebra und Logik der Relative*. B. G. Teubner, Leipzig, 1895.
- [51] Alfred Tarski. On the calculus of relations. *The Journal of Symbolic Logic*, 6(3):73–89, 1941. doi:10.2307/2268577.

- [52] Alfred Tarski and Steven Givant. *A Formalization of Set Theory without Variables*, volume 41 of *Colloquium Publications*. American Mathematical Society, Providence, RI, 1987. doi:10.1090/coll/041.
- [53] Zsolt Tuza. Applications of the set-pair method in extremal hypergraph theory. In *Extremal Problems for Finite Sets (Visegrád, 1991)*, volume 3 of *Bolyai Society Mathematical Studies*, pages 479–514. János Bolyai Mathematical Society, Budapest, 1994.
- [54] Moshe Y. Vardi. On the complexity of bounded-variable queries. In *Proceedings of the Fourteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS 1995)*, pages 266–276. ACM, 1995.
- [55] Ryan Williams. Faster decision of first-order graph properties. In *Joint Meeting of the Twenty-Third EACSL Annual Conference on Computer Science Logic and the Twenty-Ninth Annual ACM/IEEE Symposium on Logic in Computer Science (CSL-LICS)*, pages 80:1–80:6. ACM, 2014. doi:10.1145/2603088.2603121.
- [56] Triantafyllos Xylouris. On the least prime in an arithmetic progression and estimates for the zeros of Dirichlet L -functions. *Acta Arithmetica*, 150(1):65–91, 2011. doi:10.4064/aa150-1-4.

A An explicit structure

This appendix states and proves the explicit colorings announced in Section 3.1, which make the coloring of Lemma 3.1 deterministic. Nothing here is needed for the lower bounds; it is recorded so that the $|\Lambda| + 1$ structures of Corollary 3.8 can be written down rather than merely shown to exist. The symbols q , ind , ψ , ω , the modulus m , the partial coloring σ and the summation indices j, j', j'' are local to this appendix.

Proposition A.1 (Explicit colorings). *Let $N \geq 2$. A coloring with the two properties of Lemma 3.1 can be obtained in either of the following ways.*

- (a) *Keep $L = \lceil 12N^2 \ln(2N) \rceil$ and any N disjoint cells of that size. A valid coloring is then produced by a deterministic algorithm whose running time is polynomial in $|D|$.*
- (b) *In closed form: let q be a prime with $q \equiv 1 \pmod{2N}$ and $q \geq 6N^6$, let $\text{ind} : \mathbb{F}_q^\times \rightarrow \mathbb{Z}_N$ be a surjective homomorphism, identify Λ with $\mathbb{Z}_N$, and take*

$$C_\Lambda = \mathbb{F}_q^\times, \quad C_\lambda = \text{ind}^{-1}(\lambda), \quad \chi(u, v) = \text{ind}(u - v),$$

so that the cells are the cosets of the N -th powers. Here $L = (q - 1)/N$, which is $N^{O(1)}$; for $N = 2$ the coloring is the Paley graph.

Proof of Proposition A.1(a). Order the edges between cell vertices arbitrarily and color them one at a time. A *requirement* is a tuple $(\lambda, u, v, \mu, \nu)$ as in the proof of Lemma 3.1; there are fewer than $N^5 L^2$ of them. For a partial coloring σ of an initial segment of the edges, let $E(\sigma)$ be the expected number of failed requirements when the remaining edges are colored independently and uniformly at random. For a single requirement the events “ z witnesses it” concern disjoint pairs of edges for distinct z , so

$$\Pr[(\lambda, u, v, \mu, \nu) \text{ fails} \mid \sigma] = \prod_{z \in C_\lambda \setminus \{u, v\}} (1 - \Pr[z \text{ witnesses it} \mid \sigma]),$$

and each factor $\Pr[z \text{ witnesses it} \mid \sigma]$ is 0, $\frac{1}{N^2}$, $\frac{1}{N}$ or 1 according to how many of the two edges $\{u, z\}$ and $\{z, v\}$ are already colored by σ and whether those colors are μ and ν . Hence $E(\sigma)$ is computable exactly, in time $O(N^5 L^3)$.

The calculation in the proof of Lemma 3.1 shows that $E(\emptyset) < 1$. When the next edge is colored, $E(\sigma)$ is the average of its N values over the colors available for that edge, so one of those colors does not increase it. Coloring greedily in this way produces a total coloring σ^* with $E(\sigma^*) \leq E(\emptyset) < 1$; but $E(\sigma^*)$ is the number of requirements that σ^* fails, an integer, hence 0. Fewer than $N^2 L^2$ edges are colored and each step evaluates E at N candidates, so the running time is polynomial in $|D|$. Property (b) of Lemma 3.1 follows from (a) as before. $\square$

Proof of Proposition A.1(b). Primes $q \equiv 1 \pmod{2N}$ with $q \geq 6N^6$ exist by Dirichlet's theorem, and one of them is polynomial in N : applied to the modulus $m = 2N[3N^5] \geq 6N^6$, Linnik's theorem [34] — see Xylouris [56] for the best known exponent — gives a prime $q \equiv 1 \pmod{m}$ with $q = N^{O(1)}$, and any such q satisfies $q \equiv 1 \pmod{2N}$ and $q \geq m \geq 6N^6$. The modulus is taken to be m rather than $2N$ because q has to satisfy $q \geq 6N^6$ as well as $q \equiv 1 \pmod{2N}$, and for the least prime of the progression $1 \pmod{2N}$ lying above $6N^6$ there is no unconditional estimate to appeal to: at that height $2N$ is of size about $q^{1/6}$, far outside the range in which the Siegel–Walfisz theorem applies. We make no claim about that least prime, and nothing below depends on it. A surjective homomorphism $\text{ind} : \mathbb{F}_q^\times \rightarrow \mathbb{Z}_N$ exists because $N \mid q - 1$, and its fibres partition $C_\Lambda = \mathbb{F}_q^\times$ into N cells of $L = (q - 1)/N$ elements each. Since $\mathbb{F}_q^\times$ is cyclic, $\ker \text{ind}$ is its unique subgroup of index N , of order $(q - 1)/N$, and that order is even because $2N \mid q - 1$; the element -1 , the unique one of order 2, therefore lies in $\ker \text{ind}$. So $\text{ind}(-1) = 0$, hence $\chi(v, u) = \text{ind}(-1) + \chi(u, v) = \chi(u, v)$ and the coloring is symmetric.

For property (a), fix distinct $u, v \in \mathbb{F}_q^\times$, colors $\mu, \nu \in \mathbb{Z}_N$ and a cell C_γ . Any z with $\text{ind}(z) = \gamma$, $\text{ind}(z - u) = \mu$ and $\text{ind}(z - v) = \nu$ lies in $C_\gamma \setminus \{u, v\}$ and satisfies $\chi(u, z) = \mu$ and $\chi(z, v) = \nu$, as required. Put $\omega = \exp(2\pi i/N)$ and $\psi = \omega^{\text{ind}}$, a multiplicative character of $\mathbb{F}_q$ of order N , so that for $x \neq 0$ the indicator of the event $\text{ind}(x) = \lambda$ equals $\frac{1}{N} \sum_{j < N} \omega^{-j\lambda} \psi^j(x)$. The number of such z is

$$\frac{1}{N^3} \sum_{j, j', j'' < N} \omega^{-(j\gamma + j'\mu + j''\nu)} \sum_{z \notin \{0, u, v\}} \psi(z^j (z - u)^{j'} (z - v)^{j''}).$$

The triple $(j, j', j'') = (0, 0, 0)$ contributes $q - 3$. For any other triple the polynomial $X^j(X - u)^{j'}(X - v)^{j''}$ has at most three distinct roots and is not an N -th power, because $0, u, v$ are distinct and j, j', j'' are not all zero; Weil's bound for multiplicative character sums [33, Thm. 5.41] then gives at most $2\sqrt{q}$ for the sum over all of $\mathbb{F}_q$, and hence at most $2\sqrt{q} + 3$ for the sum displayed. The count is therefore at least

$$\frac{q - 3}{N^3} - 2\sqrt{q} - 3,$$

which is positive at $q = 6N^6$ for every $N \geq 2$ and increasing in q for $q > N^6$. Property (b) follows from (a). For $N = 2$ the coloring is the Paley graph on $\mathbb{F}_q$. $\square$

Using (21) with $L = (q - 1)/N$, the closed-form structure has $n_0(N) = N + q$. This is still polynomial in N , but only as $N^{O(1)}$ with the exponent supplied by Linnik's theorem, in place of the explicit $O(N^3 \log N)$ of the probabilistic construction. Only the size threshold of Section 5 changes, and it remains $2^{O(k)}$.

Neither construction brings the cells below $\Theta(N^2)$, and that is not an accident.

Remark A.2 (How small the cells can be). Property (a) of Lemma 3.1 says that for each cell C_λ the array with rows indexed by $z \in C_\lambda$, columns by $u \in C_\Lambda \setminus C_\lambda$ and entries $\chi(u, z)$, all of which are defined because $u \neq z$, is a *covering array of strength two*: any two columns take all N^2 pairs of values, since a witness supplied by (a) for two such columns lies in C_λ and is therefore distinct from both. This alone forces $L \geq N^2 + 1$. Fix a column u ; its N value classes partition the rows, and every other column must take all N values inside each class, so each class holds at least N rows and $L \geq N^2$. If $L = N^2$ then each class holds exactly N rows and each pair of values occurs exactly once, so the array is an orthogonal array of index one, with N^2 rows, N symbols and strength two; such an array has at most $N + 1$ columns, the classical bound for orthogonal arrays of index unity [10, 23], whereas this one has $(N - 1)L = (N - 1)N^2$ of them, which is more for every $N \geq 2$.

We know of no better lower bound, and Lemma 3.1 is within a logarithmic factor of this one. The factor is not forced by the covering-array condition alone: for a prime power N , concatenating four copies of an orthogonal array with N^2 rows, $N + 1$ columns, N symbols and strength two — one exists for every prime power [23] — and giving the columns distinct label vectors in $\{1, \dots, N + 1\}^4$, yields a strength-two covering array with only $4N^2$ rows and $(N + 1)^4$ columns, more than the $4N^2(N - 1)$ that are needed. Such an array does not by itself give a coloring, since χ must also be symmetric and a single χ must serve all N cells at once; we have not tried to close the gap, because L enters the results only through the threshold (21).

B Optimality of the interpretations

Corollary 3.8 converts detectable colors into a lower bound, so the larger Λ_0 is, the better. This appendix bounds $|\Lambda_0|$ from above, for every set of colors and every type-constant interpretation, and shows that the interpretations (12) and (15) of Section 4 are optimal in this sense. Nothing here is needed for Theorems 1.2 and 1.3; the results explain why Theorem 1.2 carries a central binomial coefficient and why the factor of two in Theorem 1.3 cannot be removed by a better interpretation. The notation is that of Sections 3 and 4.

B.1 Where a deletion can be seen

The lemma below bounds $|\Lambda_0|$ from above, for every set of colors and every type-constant interpretation. It turns the informal account of the anchors in Section 1.4.2 into a precise statement, and Appendices B.3 and B.4 combine it with the form of the clauses of Ψ_k and Φ_k .

Call $\varphi(x, y) = \exists z \psi$ a *witness query* if ψ is quantifier-free and every atom of ψ , equalities included, has z as exactly one of its two arguments and x or y as the other. Both Ψ_k and Φ_k are witness queries, and so is the Φ'_k of Remark B.5.

Lemma B.1 (Where a deletion can be seen). *Let $\varphi = \exists z \psi$ be a witness query whose relation symbols are interpreted on $\mathfrak{M}$ by type-constant relations. Every color detectable for φ is detected at a pair one of whose endpoints is the hub, and*

- (a) *for each anchor x , at most one color is detected at (x, h) and at most one at (h, x) ;*
- (b) *at most one color is detected at (h, h) ; at most one at the pairs (h, v) with v a cell vertex, taken together; and at most one at the pairs (u, h) with u a cell vertex;*
- (c) *these three kinds detect at most two colors between them.*

Hence at most $2|X| + 2$ colors are detectable for φ on $\mathfrak{M}$.

Proof. Every atom of ψ is evaluated at (u, z) , (z, u) , (z, v) or (v, z) , and the types of all four are determined by $(\text{tp}(u, z), \text{tp}(z, v))$, since converse carries each type onto a type (Corollary 3.3). The interpretations being type-constant, the truth value of ψ at a witness z therefore depends only on the profile of z at (u, v) ; write $\psi(\tau_1, \tau_2)$ for it. For $u, v \in D_J$, separate the cell witnesses from the remaining witnesses:

$$\varphi^{\mathfrak{M} \upharpoonright J}(u, v) = c(u, v) \vee \sigma_J(u, v), \quad \sigma_I(u, v) = \bigvee_{z \in C_I \setminus \{u, v\}} \psi(\text{tp}(u, z), \text{tp}(z, v)),$$

where $c(u, v)$ collects the witnesses in $X \cup \{h\} \cup (\{u, v\} \cap U)$. These survive every deletion and keep their profiles, so c does not depend on J . As σ_I is monotone in I , detecting λ at (u, v) requires $c(u, v) = 0$, $\sigma_\Lambda(u, v) = 1$ and $\sigma_{\Lambda \setminus \{\lambda\}}(u, v) = 0$; only the last two are used below.

The profiles realized by the cell witnesses were listed in the proof of Lemma 3.4, and among the entries of (11) only the five carrying ℓ_I , r_I or d_I depend on I ; at every other pair, $\sigma_I(u, v) = \sigma_\Lambda(u, v)$ for every nonempty I , which proves the first assertion. Those five kinds of pairs are (x, h) and (h, x) for an anchor x , the pair (h, h) , and the pairs (h, v) and (u, h) with v and u cell vertices, and at them the profiles realized by $z \in C_I$ are respectively

$$(\tau_{x,*}, \tau_\lambda), \quad (\tau_\lambda, \tau_{*,x}), \quad (\tau_\lambda, \tau_\lambda), \quad (\tau_\lambda, \tau_\mu), \quad (\tau_\mu, \tau_\lambda) \quad (\lambda \in I, \mu \in \Lambda),$$

where the last two follow from Lemma 3.1(b), which supplies every μ on the cell side inside every retained cell. In each case $\sigma_I(u, v) = \bigvee_{\lambda \in I} s(\lambda)$ for a function s of λ alone, and in the last two cases s depends on neither v nor u (the term c does depend on them, since the hub and the endpoint are themselves witnesses there, but c is unaffected by deletions). Hence $\sigma_\Lambda = 1$ and $\sigma_{\Lambda \setminus \{\lambda\}} = 0$ force $s(\lambda) = 1$ and $s(\mu) = 0$ for every $\mu \neq \lambda$, which at most one λ can satisfy; in the last two cases that single λ serves every choice of the cell vertex at once. This proves (a) and (b).

For (c), suppose that (h, h) detects λ_0 while some pair (h, v) detects a different color λ_1 . The first gives $\psi(\tau_{\lambda_0}, \tau_{\lambda_0}) = 1$, the second gives $\bigvee_{\mu \in \Lambda} \psi(\tau_{\lambda_0}, \tau_\mu) = 0$, and these contradict each other. Hence (h, h) and the pairs (h, v) detect the same color if they detect any color at all, and the same holds for (h, h) and the pairs (u, h) by symmetry. Thus if (h, h) detects a color, the three kinds together detect exactly one color, and otherwise they detect at most two. Summing $|X| + |X| + 2$ gives the bound. $\square$

The bound is linear in the number of anchors, which is the precise content of the informal remark in Section 1.4.2 that the anchors are the resource that the lower bound consumes. The constant is exact: for $|\Lambda| \geq 2|X| + 2$, take one relation symbol per type, interpreted by the indicator of that type, and let ψ be the disjunction, over the profiles $(\tau_{\lambda_0}, \tau_{\lambda_1})$ and, for each anchor x , $(\tau_{x,*}, \tau_{\alpha_x})$ and $(\tau_{\beta_x}, \tau_{*,x})$, where the $2|X| + 2$ colors $\lambda_0, \lambda_1, \alpha_x, \beta_x$ are distinct, of the symbol of the first type applied to (x, z) conjoined with the symbol of the second type applied to (z, y) . Then λ_0 is detected at the pairs (h, v) , λ_1 at the pairs (u, h) , and α_x and β_x at (x, h) and (h, x) , respectively (the witnesses in $X \cup \{h\} \cup \{u, v\}$ realize none of these profiles, so the term c of the proof vanishes there).

B.2 Bollobás's set-pair inequality

The bound for Ψ_k comes from a classical inequality of extremal set theory, which we state and prove for completeness. A finite family $(A_\lambda, B_\lambda)_{\lambda \in I}$ of pairs of sets is *cross-intersecting* if $A_\lambda \cap B_\lambda = \emptyset$ for every $\lambda \in I$ and $A_\lambda \cap B_\mu \neq \emptyset$ for all $\lambda \neq \mu$ in I .

Lemma B.2 (Bollobás’s set-pair inequality [3]). *Every cross-intersecting family $(A_\lambda, B_\lambda)_{\lambda \in I}$ satisfies*

$$\sum_{\lambda \in I} \binom{|A_\lambda| + |B_\lambda|}{|A_\lambda|}^{-1} \leq 1. \quad (25)$$

In particular, if all the sets are subsets of $[k]$, then $|I| \leq \binom{k}{\lfloor k/2 \rfloor}$.

Proof. Let π be a uniformly random linear order of the union of all the sets, and let E_λ be the event that every element of A_λ precedes every element of B_λ . The two sets being disjoint, $\Pr[E_\lambda]$ is the reciprocal of $\binom{|A_\lambda| + |B_\lambda|}{|A_\lambda|}$. The events are pairwise disjoint: for $\lambda \neq \mu$ choose $a \in A_\lambda \cap B_\mu$ and $b \in A_\mu \cap B_\lambda$, both available, and distinct because $a = b$ would put a in $A_\lambda \cap B_\lambda = \emptyset$; then E_λ puts a before b while E_μ puts b before a . Summing the probabilities gives (25). For subsets of $[k]$, disjointness gives $|A_\lambda| + |B_\lambda| \leq k$, so every summand is at least $\binom{k}{\lfloor k/2 \rfloor}^{-1}$, and the second claim follows. $\square$

The inequality is usually quoted in the uniform case $|A_\lambda| = a$ and $|B_\lambda| = b$, where it reads $|I| \leq \binom{a+b}{a}$; the proof above is the standard permutation argument (cf. Katona [30]), and Tuza [53] surveys the many uses of the inequality. The bound $\binom{k}{\lfloor k/2 \rfloor}$ is attained by letting B_λ range over all $\lfloor k/2 \rfloor$ -element subsets of $[k]$ and taking $A_\lambda = [k] \setminus B_\lambda$.

B.3 The middle layer is forced

Fix such an interpretation and write

$$G(\tau) = \{i \in [k] : R_i(\tau) = 0\}$$

for the set of clauses that a type τ leaves unsatisfied. A witness with profile (τ_1, τ_2) satisfies clause i exactly when $R_i(\tau_1) = 1$ or $R_i(\tau_2) = 1$, so it satisfies all k clauses exactly when

$$G(\tau_1) \cap G(\tau_2) = \emptyset. \quad (26)$$

Because each clause is a disjunction, the test has become a disjointness test between two subsets of $[k]$, symmetric in the two edges. This is the only feature of Ψ_k used below.

Proposition B.3 (The middle layer is forced). *Let Λ be any finite set of colors with $|\Lambda| \geq 2$, let X be any finite set of anchors, and interpret $R_1, \dots, R_k$ on the resulting structure $\mathfrak{M}$ by any type-constant relations. The set Λ_0 of colors detectable for Ψ_k satisfies*

$$|\Lambda_0| \leq \binom{k}{\lfloor k/2 \rfloor},$$

and the interpretation (12) attains this bound.

Proof. Ψ_k is a witness query, so Lemma B.1 applies: every detectable color is detected at one of the five kinds of pairs listed there. Let Λ'_0 be the set of colors detected at a pair (x, h) or (h, x) with x an anchor. We first bound $|\Lambda'_0|$ and then show that the three kinds of pairs not involving an anchor add nothing to it.

Fix $\lambda \in \Lambda'_0$ together with a pair at which it is detected. If that pair is (x, h) then, by the proof of Lemma B.1, the cell witnesses realize the profiles $(\tau_{x,*}, \tau_\mu)$ with μ retained, so detection means that (26) holds for $(\tau_{x,*}, \tau_\lambda)$ and fails for $(\tau_{x,*}, \tau_\mu)$ for every color $\mu \neq \lambda$; put $A_\lambda = G(\tau_{x,*})$. If the pair is (h, x) the profiles are $(\tau_\mu, \tau_{*,x})$ and the same holds with $A_\lambda = G(\tau_{*,x})$, because (26) is symmetric. Either way, writing $B_\lambda = G(\tau_\lambda)$,

$$A_\lambda \cap B_\lambda = \emptyset, \quad A_\lambda \cap B_\mu \neq \emptyset \quad (\mu \in \Lambda, \mu \neq \lambda).$$

In particular $(A_\lambda, B_\lambda)_{\lambda \in \Lambda'_0}$ is a cross-intersecting family of subsets of $[k]$, and Lemma B.2 gives $|\Lambda'_0| \leq \binom{k}{\lfloor k/2 \rfloor}$.

Now consider the pairs not involving an anchor. At a pair (h, v) with v a cell vertex, the proof of Lemma B.1 gives $s(\lambda) = \bigvee_{\mu \in \Lambda} [G(\tau_\lambda) \cap G(\tau_\mu) = \emptyset]$, and detection of λ requires $s(\lambda) = 1$ and $s(\mu) = 0$ for every $\mu \neq \lambda$. Since (26) is symmetric, the color witnessing $s(\lambda) = 1$ can only be λ itself, so $G(\tau_\lambda) = \emptyset$; but then $s(\mu) = 1$ for every μ , a contradiction because $|\Lambda| \geq 2$. Hence no color is detected at the pairs (h, v) and, by symmetry, none at the pairs (u, h) . At (h, h) , a color λ is detected only if $G(\tau_\lambda) = \emptyset$ and $G(\tau_\mu) \neq \emptyset$ for every $\mu \neq \lambda$. In that case $G(\tau) \cap G(\tau_\lambda) = \emptyset$ for every type τ , so no anchor pair detects a color other than λ , and $\Lambda_0 = \{\lambda\}$. In all cases $|\Lambda_0| \leq \max(1, |\Lambda'_0|) \leq \binom{k}{\lfloor k/2 \rfloor}$. For (12) we have $G(\tau_{x_A,*}) = [k] \setminus A$ and $G(\tau_B) = B$, so (26) reads $B \subseteq A$, and (13) makes all $\binom{k}{\lfloor k/2 \rfloor}$ colors detectable. $\square$

The interpretation (12) is precisely the extremal family described after Lemma B.2: its pairs are $([k] \setminus A, A)$ for $A \in \Lambda$. The loss of a factor $\Theta(\sqrt{k})$ is therefore inherent to Ψ_k together with this construction (Section 1.4.6). The two extra colors allowed by Lemma B.1 never materialize for Ψ_k , because the symmetry of (26) prevents the pairs (h, v) and (u, h) from detecting anything; they are available only to witness queries with an asymmetric ψ , such as the one exhibited after that lemma. Note that the proposition bounds the number of colors that the structure can be made to detect; it is not an upper bound on the number of compositions that Ψ_k requires, which we do not determine.

It is instructive to see what the disjunction costs. Detection at an anchor pair already forces the tuples $(R_i(\tau_\lambda))_{i \leq k}$ to be pairwise distinct over Λ'_0 , which by itself gives only $|\Lambda'_0| \leq 2^k$; it is the disjointness test (26) that reduces 2^k to the middle layer. The next subsection replaces the disjunctive clause by one that tests two labels for equality, a test subject to no such constraint, and thereby recovers all 2^k subsets.

B.4 All labels at once

Proposition B.3 bounded the number of colors that Ψ_k can be made to detect by the size of the middle layer. For Φ_k the corresponding bound is 2^k , so (15) gives up nothing; the proof is shorter than that of Proposition B.3, because the agreement test (14) is already a test between two sets.

Fix a type-constant interpretation of the $2k$ symbols, over any set of colors and with any number of anchors, and give a type τ the *key set*

$$\mathcal{S}_\tau = \{A \subseteq [k] : H_A(\tau) = 1\},$$

where H_A is the relation of (17); being an intersection of type-constant relations, H_A is type-constant, so the key set is well defined. Explicitly, $A \in \mathcal{S}_\tau$ means that $P_i(\tau) = 1$ for every $i \in A$ and $Q_i(\tau) = 1$ for every $i \notin A$. A witness with profile (τ_1, τ_2) satisfies clause i exactly when P_i holds on both types

or Q_i does; recording for each i which alternative is taken, and collecting the indices of the first kind into a set A , we see that it satisfies all k clauses exactly when

$$\mathcal{S}_{\tau_1} \cap \mathcal{S}_{\tau_2} \neq \emptyset. \quad (27)$$

This is the counterpart of (26), and the contrast between the two conditions is the whole difference between the two formulas: there the test is disjointness, here it is non-empty intersection. The key set also refines the dichotomy between labelled and unlabelled types of Section 4.2: it is empty exactly when τ carries no label, a singleton exactly when τ carries a label, and larger when some index has both of its symbols true, a case that the dichotomy does not cover but that a general interpretation may use. Finally, the union in (17) ranges over exactly the sets A that can occur in a key set, and (27) explains why: a witness serves exactly when a single H_A holds on both of its edges.

Proposition B.4 (All labels at once). *Let Λ be any finite set of colors with $|\Lambda| \geq 2$, let X be any finite set of anchors, and interpret $P_1, Q_1, \dots, P_k, Q_k$ on the resulting structure $\mathfrak{M}$ by any type-constant relations. The set Λ_0 of colors detectable for Φ_k satisfies*

$$|\Lambda_0| \leq 2^k,$$

and the interpretation (15) attains this bound.

Proof. Φ_k is a witness query, so Lemma B.1 applies: every detectable color is detected at one of the five kinds of pairs listed there. As in the proof of Proposition B.3, let Λ'_0 be the set of colors detected at a pair (x, h) or (h, x) with x an anchor.

Fix $\lambda \in \Lambda'_0$ together with a pair at which it is detected. If that pair is (x, h) then, by the proof of Lemma B.1, the cell witnesses realize the profiles $(\tau_{x,*}, \tau_\mu)$ with μ retained, so detection means that (27) holds for $(\tau_{x,*}, \tau_\lambda)$ and fails for $(\tau_{x,*}, \tau_\mu)$ for every $\mu \neq \lambda$; put $T_\lambda = \mathcal{S}_{\tau_{x,*}}$. If the pair is (h, x) the profiles are $(\tau_\mu, \tau_{*,x})$ and the same holds with $T_\lambda = \mathcal{S}_{\tau_{*,x}}$, because (27) is symmetric. Either way

$$T_\lambda \cap \mathcal{S}_{\tau_\lambda} \neq \emptyset, \quad T_\lambda \cap \mathcal{S}_{\tau_\mu} = \emptyset \quad (\mu \in \Lambda, \mu \neq \lambda).$$

Choosing for each $\lambda \in \Lambda'_0$ a set in $T_\lambda \cap \mathcal{S}_{\tau_\lambda}$ therefore defines a map $\Lambda'_0 \rightarrow 2^{[k]}$, and it is injective: the set chosen for λ lies in T_λ , hence in no $\mathcal{S}_{\tau_\mu}$ with $\mu \neq \lambda$, whereas the set chosen for μ lies in $\mathcal{S}_{\tau_\mu}$. Hence $|\Lambda'_0| \leq 2^k$.

The pairs not involving an anchor again add nothing. At a pair (h, v) with v a cell vertex, $s(\lambda) = \bigvee_{\mu \in \Lambda} [\mathcal{S}_{\tau_\lambda} \cap \mathcal{S}_{\tau_\mu} \neq \emptyset]$, and detection of λ requires $s(\lambda) = 1$ and $s(\mu) = 0$ for every $\mu \neq \lambda$. As (27) is symmetric, the color witnessing $s(\lambda) = 1$ can only be λ itself, so $\mathcal{S}_{\tau_\lambda} \neq \emptyset$, while $s(\mu) = 0$ forces $\mathcal{S}_{\tau_\mu} = \emptyset$ for every $\mu \neq \lambda$ (take μ itself in the join). The same two conditions are necessary for detection at (h, h) and, by symmetry, at the pairs (u, h) . Under these conditions an anchor pair can detect a color λ' only if $\mathcal{S}_{\tau_{\lambda'}} \neq \emptyset$, that is, only if $\lambda' = \lambda$, so $\Lambda_0 = \{\lambda\}$. In all cases $|\Lambda_0| \leq \max(1, |\Lambda'_0|) \leq 2^k$.

For (15), a type carrying the label A has $\mathcal{S}_\tau = \{A\}$, while a type carrying no label has $c_i(\tau) = e_\perp$ for some i and hence $\mathcal{S}_\tau = \emptyset$. So (27) holds for $(\tau_{x_A,*}, \tau_B)$ exactly when $A = B$, and fails whenever one of the two types is unlabelled; this is (16), and all 2^k colors are detectable. $\square$

Together, the two propositions locate the slack that remains in Theorem 1.3. Corollary 3.8 turns $|\Lambda_0|$ detectable colors into $\lceil |\Lambda_0|/2 \rceil$ compositions, so on this structure no interpretation of

the symbols of Φ_k yields a bound above the 2^{k-1} of Theorem 1.3, against the 2^k compositions of (17). Unlike the factor $\Theta(\sqrt{k})$ of Section 4.1, the factor of two is therefore not a loss incurred in the choice of labels: it is the support number $\text{sn}(\wedge) = 2$ of Lemma 3.6, that is, the ability of a single composition to monitor two colors at once. This value is exact: the family exhibited in the proof of Lemma 3.6 needs both of its colors, so the per-gate bound behind $|K| \leq 2m$ cannot be improved, although a particular composition may have a smaller support. Closing the factor of two would require a different invariant, not a different interpretation.

B.5 The equivalence query

One variant of Φ_k is worth recording, because it looks simpler than Φ_k and is not covered by Theorem 1.3.

Remark B.5 (The equivalence query). Over k symbols let $\Phi'_k(x, y) = \exists z \bigwedge_{i=1}^k (R_i(x, z) \leftrightarrow R_i(z, y))$, with $\leftrightarrow$ an abbreviation of fixed size, so that $|\Phi'_k| = \Theta(k)$. Substituting $P_i := R_i$ and $Q_i := \overline{R_i}$ turns a term for Φ_k into a term for Φ'_k with the same number of compositions, so Theorem 1.3 does *not* transfer to Φ'_k ; a separate interpretation is needed, and it costs one label, because an equivalence is also satisfied by two false bits. Write $\text{prof}(\tau) = \{i : R_i(\tau) = 1\}$ for the *profile* of a type, take $\Lambda = 2^{[k]}$ with an anchor x_A for each *nonempty* A , and put $\text{prof}(\tau_{x_A, *}) = A$, $\text{prof}(\tau_B) = B$, $\text{prof}(\tau_{=}) = \emptyset$, $\text{prof}(\tau_{x_A, x_B}) = [k] \setminus B$ and $\text{prof}(\tau_{*, x_B}) = \emptyset$. At (x_A, h) a cell witness in C_B succeeds iff $B = A$; the hub shows $\emptyset$ on its loop, which is no anchor's label; and an anchor x_B is seen from x_A with profile $[k] \setminus B$ while showing B to the hub, and no set equals its complement. So the $2^k - 1$ nonempty colors are detectable, $2m \geq 2^k - 1$, and hence $m \geq 2^{k-1}$ again because m is an integer. Unlike Φ_k , the formula Φ'_k is not positive.

C Further kernels and scalar interactions

This appendix collects three remarks on the support number that are not needed for the theorems of Section 6: the upper bound of Proposition 6.1 is attained over the common tropical semirings, wider scalars reduce the number of products by exactly a factor of their width, and kernels of unbounded support number exist. The notation is that of Sections 3.3 and 6.

C.1 The support number of tropical multiplication

The upper bound $\text{sn}(\otimes) \leq 3\beta$ of Proposition 6.1 is attained whenever the semiring has room for it. By the proof of that proposition, a support has to fix the three scalar values $\bigvee_{\lambda \in J} p_\lambda$, $\bigvee_{\lambda \in J} q_\lambda$ and $\bigvee_{\lambda \in J} p_\lambda \otimes q_\lambda$. Suppose that $\mathbb{S}$ has breadth one and contains elements $0_{\mathbb{S}} < a < b$ with $a \otimes a \neq 0_{\mathbb{S}}$, where $a \leq b$ means $a \vee b = b$ (for min-plus this is the reverse of the numerical order); the letters a and b are local to this paragraph. Max-plus, min-plus and max-min, tabulated in Section 6.2, all qualify, with $(a, b) = (0, 1)$, $(1, 0)$ and $(\frac{1}{2}, 1)$ respectively. Take three colors with

$$(p_{\lambda_1}, q_{\lambda_1}) = (b, 0_{\mathbb{S}}), \quad (p_{\lambda_2}, q_{\lambda_2}) = (0_{\mathbb{S}}, b), \quad (p_{\lambda_3}, q_{\lambda_3}) = (a, a).$$

The two marginal joins are both b because $0_{\mathbb{S}} < a < b$, and the paired response is $a \otimes a$ because $0_{\mathbb{S}}$ annihilates. Only λ_1 attains the first marginal join, because $0_{\mathbb{S}} \vee a = a \neq b$; only λ_2 attains the second; and only λ_3 makes the paired response differ from $0_{\mathbb{S}}$. Every support therefore contains

all three colors, and together with the proposition this gives $\text{sn}(\otimes) = 3$. The three responses are independent here: $\{\lambda_3\}$ fixes the paired response but neither marginal join, while $\{\lambda_1, \lambda_2\}$ fixes both marginal joins but not the paired response. Over these semirings, the factor 3β in Theorems 6.2 and 6.4 is thus the support number itself and not slack in the proposition; closing it would require a different invariant, as with the factor of two in Section 4.2.

C.2 Vector-valued scalars, and other scalar interactions

The same invariant also separates scalar capacity from the number of aggregations. Take $S = \mathbb{B}^b$, the b -bit scalars, with coordinatewise Boolean operations; embed an input bit α diagonally as $(\alpha, \dots, \alpha)$, and require the same encoding of the output. A single product then performs b Boolean products, and pointwise functions may mix coordinates freely.

Widening the scalars gains a factor of b , and nothing more.

Proposition C.1 (Bits per scalar against product count). *For $k, b \geq 1$, the minimum number m of these products computing Φ_k satisfies*

$$\frac{2^{k-1}}{b} \leq m \leq \left\lceil \frac{2^k}{b} \right\rceil.$$

The bounds hold with or without sharing, and at every fixed size $n \geq n_0(2^k)$ for size-specific targets.

Proof. Each coordinate of a reference product has a support of at most two colors, so their union has size at most $2b$, and the same support preserves the whole tuple, including through pointwise maps that mix coordinates. The read-off identity (16) therefore forces $2bm \geq 2^k$, by restriction or by recoloring at a fixed size, which is the lower bound.

For the upper bound, partition the 2^k subsets A into groups of size at most b . A pointwise map puts the values H_A of (17) into separate coordinates, with zeros in the unused ones, and one product then computes all the H_A ; H_A of a group. Pointwise disjunction merges the coordinates and the groups and copies the result diagonally. Product values need not be shared. The target here is the encoded Boolean query Φ_k , not F_k on arbitrary $\mathbb{B}^b$ -valued inputs. $\square$

Beyond semiring multiplication. For $S = [0, 1]$ with join $\max$, any kernel that is monotone in each argument, in a direction that does not depend on the value of the other argument, has support number at most three: choose one color attaining the appropriate extreme of p_λ , which serves the entire left response, one color attaining the appropriate extreme of q_λ , which serves the entire right response, and one color maximizing the paired response. The two arguments may have different directions of monotonicity; what the uniformity provides is that a single color serves the whole of ℓ_λ , and a single color the whole of r_λ . Consequently, the Boolean queries require at least $N(\varphi)/3$ such aggregations, even with different kernels at different gates and arbitrary uniform pointwise numerical functions. This follows from the same preservation theorem and uses no associative multiplication. One example is the comparison kernel, which is 1 if $p \leq q$ and 0 otherwise: it is nonincreasing in p and nondecreasing in q , regardless of the other argument.

Unrestricted kernels can behave differently. On the nonnegative integers with join $\max$, the kernel given by $p \odot q = 1$ if $p \& q \neq 0$ and $p \odot q = 0$ otherwise, where $\&$ denotes bitwise AND, has unbounded support number, since distinct one-bit masks give arbitrarily large irredundant left

responses. For Φ_k , a free pointwise map encodes at each entry (u, v) the set of subsets $A \subseteq [k]$ with $(u, v) \in H_A$, where H_A is the intersection of (17), as the bit mask $\sum_{A:(u,v) \in H_A} 2^{\text{num } A}$ with $\text{num } A = \sum_{i \in A} 2^{i-1}$. Distinct subsets occupy distinct bit positions, so two masks have a nonzero bitwise AND exactly when some H_A contains both entries; a single aggregation with this kernel therefore computes $\bigcup_{A \subseteq [k]} H_A$, which is Φ_k by (17). On the structures of Section 4.2, the mask is $2^{\text{num } A}$ at an entry whose type carries the label A and 0 at an unlabelled entry. Unlimited scalar precision is therefore not ruled out by itself: what a finite support number limits is the way in which encoded states can interact through a witness.